Lab Flow: Programming Laboratory Management System

Mini Project Report

Submitted by

ALLEN JOHN JOY

REG NO: FIT25MCA-2008

Submitted in partial fulfilment of the requirements for the award of the degree of

Master of Computer Applications

Of

A P J Abdul Kalam Technological University



FEDERAL INSTITUTE OF SCIENCE AND TECHNOLOGY (FISAT)® ANGAMALY-683577, ERNAKULAM(DIST)

OCTOBER 2026

DECLARATION

I, **ALLEN JOHN JOY** hereby declare that the report of this project work, submitted to the Department of Computer Applications, Federal Institute of Science and Technology (FISAT), Angamaly in partial fulfillment of the award of the degree of Master of Computer Applications is an authentic record of my original work.

The report has not been submitted for the award of any degree of this university or any other university.

Date:

Place:

FEDERAL INSTITUTE OF SCIENCE AND TECHNOLOGY (FISAT)

ANGAMALY, ERNAKULAM-683577

DEPARTMENT OF COMPUTER APPLICATIONS



CERTIFICATE

This is to certify that the project report titled "**LabFlow: Programming Laboratory Management System**" submitted by **ALLEN JOHN JOY** (Roll No. FIT25MCA-2008) towards partial fulfillment of the requirements for the award of the degree of Master of Computer Applications is a record of bonafide work carried out by him during the year 2026.

Project Guide
Dr. Rose Mary Mathew

Head of the Department
Dr. Shahna K U

ACKNOWLEDGEMENT

I express my deep gratitude to Dr. Jacob Thomas V, Principal, FISAT, for providing the necessary facilities and support that enabled me to complete my mini project successfully.

I extend my sincere thanks to Dr. Shahna K U, Head of the Department of Computer Applications, FISAT, for her inspiration and encouragement throughout the project.

I express my heartfelt gratitude to Dr. Rose Mary Mathew, my Project Guide, for her valuable suggestions, guidance and support at every stage of the project.

I am also grateful to Dr. Deepa Mary Mathews, my Scrum Guide, for her constant encouragement and unwavering support throughout the project. I extend my sincere thanks to all the faculty and lab faculty members of the department for their guidance and assistance.

Finally, I thank my friends for their valuable suggestions, cooperation, and encouragement, which contributed to the successful completion of this project.

ABSTRACT

Programming laboratories are the backbone of practical learning in the MCA programme, yet in many institutions they are still run through paper registers, shared folders and disconnected tools. Exercises are circulated manually, students run code on whichever machine is available, submissions arrive as loose files, attendance is marked on paper and evaluation is difficult to track. LabFlow is a web-based Programming Laboratory Management System developed for the Department of Computer Applications, FISAT, which brings all of these activities onto a single platform for students, faculty and administrators.

The main contribution of LabFlow is an in-browser Integrated Development Environment built on the Monaco editor. Students write, run and test programs in C, Java and Python directly in the browser, with custom input and instant output, without installing any compiler. Code is executed by a server-side engine that uses the compilers and interpreters of the host with time limits and output limits, and falls back to the Judge0 sandbox when required. The second major contribution is GitHub synchronisation: a student connects GitHub once through OAuth, after which every submission is committed automatically to a private repository that LabFlow creates for that laboratory, so that laboratory work grows into a version-controlled portfolio.

Around these two features LabFlow provides Google sign-in with JSON Web Token based role access, a student portal with exercises, submissions and attendance, a faculty portal for assigning exercises, defining visible and hidden test cases, evaluating submissions and marking attendance, timed online exams with automatic grading and focus-violation monitoring, and an administration console with announcements, audit log and maintenance mode. The system is implemented with React, FastAPI and MongoDB and exposes more than 80 REST endpoints. Measurements of the execution engine show results in well under one second for Python and C and under one second for Java, and the working system has been verified through all three portals.

Contents

1 INTRODUCTION **1**

2 LITERATURE SURVEY **4**

3 IMPLEMENTATION **7**

3.1 System Architecture 7

3.1.1 Methodology 14

3.1.2 Hardware Requirements 14

3.1.3 Software Requirements 15

3.2 Database Design 15

3.3 Modules 18

3.3.1 AUTHENTICATION AND ACCESS CONTROL 18

3.3.2 STUDENT PORTAL 19

3.3.3 IN-BROWSER IDE AND CODE EXECUTION ENGINE 19

3.3.4 GITHUB INTEGRATION AND SYNCHRONISATION 22

3.3.5 SUBMISSION AND EVALUATION 23

3.3.6 ATTENDANCE 24

3.3.7 ONLINE EXAMS 25

3.3.8 FACULTY PORTAL 25

3.3.9 ADMINISTRATION PORTAL 26

3.3.10 UML Diagrams 27

3.3.11 Issues Faced and Remedies Taken 30

3.4 ALGORITHM 30

4 RESULT ANALYSIS **35**

4.1 Testing, Test Cases and Test Results 35

4.2 Functional Results 36

4.3 Code Execution Performance 36

4.4 Comparison with the Manual Laboratory Process 38

4.5 Discussion 38

5 CONCLUSION AND FUTURE SCOPE **39**

5.1 Conclusion 39

5.2 Future Scope 39

6 CODING **41**

6.1 Source Code 41

6.1.1 main.py 41

6.1.2 jwt_service.py 44

6.1.3 security.py 45

6.1.4 local_execution_service.py 46

7 SCREEN SHOTS 48

8 REFERENCES 54

Chapter 1
Introduction

1.1 Background

Programming laboratories form the practical core of the Master of Computer Applications (MCA) curriculum. In a typical semester a student attends several laboratories such as Network Security and Applications, Advanced Database Management Systems and Object Oriented Programming (Java), each with its own faculty member, timetable, list of exercises and evaluation scheme. In many institutions these activities are still coordinated through paper records, shared folders and messaging groups. Exercises are circulated manually, students write programs on whichever machine is free, submissions arrive as loose files, attendance is marked on paper and the faculty has no single place in which to see who has submitted what.

LabFlow is a web-based Programming Laboratory Management System built for the Department of Computer Applications, Federal Institute of Science and Technology (FISAT). It brings the complete laboratory workflow onto one platform that serves three kinds of users: students, faculty members and administrators. Its distinguishing feature is an in-browser Integrated Development Environment (IDE) that lets a student write, run and test programs in C, Java and Python without installing any software, together with a GitHub integration that stores every submission in the student's own repository automatically.

1.2 Problem Statement

The existing way of conducting programming laboratories has several practical difficulties:

- Students need a properly configured compiler or interpreter on every machine they use, and set-up problems consume lab time.
- Exercises, due dates, syllabi and announcements are spread across notice boards, messages and printed manuals.
- Submissions are collected as files, which makes it hard for faculty to check completion, give feedback and keep an evaluation history.
- Attendance is recorded manually and is not connected to the laboratory timetable.
- Students rarely keep an organised, version-controlled record of their laboratory work that they can later show as a portfolio.

- Administrators have no consolidated view of students, faculty allocation, enrolments and system activity.

A single integrated platform that addresses these difficulties, and that works from an ordinary web browser, is therefore needed.

1.3 Objectives

- To provide secure, role-based access for students, faculty and administrators using institutional Google sign-in and token-based authentication.
- To build an in-browser IDE with a professional code editor, standard input support and instant output for C, Java and Python.
- To execute student code safely on the server with time limits, output limits and a sandbox fallback.
- To synchronise each submission to a private GitHub repository created and maintained automatically for the student.
- To let faculty assign exercises, define visible and hidden test cases, review submissions and award marks with feedback.
- To record laboratory attendance against the timetable and present it to both students and faculty.
- To conduct timed online exams with automatic grading of multiple-choice questions and monitoring of focus violations.
- To give administrators tools for managing classes, students, faculty, enrolments, announcements, audit logs and maintenance.

1.4 Scope of the Project

LabFlow currently serves the MCA Semester 3 laboratory term and is configured with three laboratories: Network Security and Applications (20MCA131), Object Oriented Programming Lab in Java (20MCA132) and Advanced DBMS Lab (20MCA134). The platform supports the programming languages C, Java and Python, and is designed so that further laboratories, classes and languages can be added through the administration portal and configuration. The system consists of a React single-page web application and a FastAPI back end with a MongoDB database, exposing more than 80 REST endpoints.

1.5 Technology Stack

Table 1.1: Technologies used in LabFlow

Layer	Technology and purpose
Front end	React 19 with React Router 7 for the single-page interface; Vite 8 as the build tool; Tailwind CSS 4 for styling; Radix UI primitives, Framer Motion, Sonner (notifications) and Lucide icons.
Code editor	Monaco Editor (the editor that powers Visual Studio Code) through @monaco-editor/react, with C, Java and Python language modes.
Back end	Python with FastAPI and Uvicorn; Pydantic for validation; HTTPX for outgoing HTTP calls.
Database	MongoDB accessed asynchronously through Motor and PyMongo.
Authentication	Google OAuth 2.0 / OpenID Connect through Authlib; JSON Web Tokens (python-jose, HS256); PBKDF2-HMAC-SHA256 password hashing.
Code execution	Local compiler / interpreter runner (gcc, javac / java, Python) with Judge0 as the remote sandbox fallback.
GitHub integration	GitHub OAuth application and REST API (repositories and contents); Fernet symmetric encryption (cryptography library) for stored tokens.
Testing	End-to-end flow scripts for the admin, faculty, student, IDE, execution, submission and attendance modules.

1.6 Development Approach

The project was developed incrementally following the Scrum framework [7]. Work was organised in phases that built on one another: the portals and authentication first, then the IDE and the code execution engine, followed by submissions and evaluation, attendance, GitHub integration, online exams and administration features. The sprint-wise planning and review records are maintained separately in the Scrum Book.

1.7 Organisation of the Report

Chapter 2 reviews related work and existing platforms. Chapter 3 describes the implementation: architecture, database design, modules and key algorithms. Chapter 4 presents the results of the work. Chapter 5 concludes the report and outlines future scope. Chapter 6 lists the important source code, Chapter 7 shows screenshots of the system and Chapter 8 lists the references.

Chapter 2

Literature Survey

2.1 Automatic Assessment of Programming Work

Automatic assessment of programming exercises has been studied for several decades. Douce, Livingstone and Orwell reviewed test-based assessment systems and described how submitted programs can be compiled and executed against predefined test data so that results are produced consistently and quickly [2]. Ihantola et al. later reviewed systems developed between 2005 and 2010, comparing how they handle security, feedback, language support and integration with learning environments [1]. Both reviews highlight three recurring needs: safe execution of untrusted code, meaningful feedback for the learner, and a workflow that fits the way courses are actually taught. LabFlow applies these ideas directly through its time-limited execution engine, visible and hidden test cases, and faculty evaluation with feedback.

2.2 Existing Platforms

Several categories of tools are used for programming education today. Learning management systems such as Moodle and Google Classroom manage courses, assignments and grades, but code is normally written and run elsewhere and uploaded as a file. Online judges and coding platforms such as HackerRank and CodeChef provide in-browser editors and automatic test evaluation, but they are general-purpose services and do not model an institution's laboratory timetable, attendance, faculty allocation or lab-specific exercise assignment. GitHub Classroom connects assignments to GitHub repositories, which is valuable for version control, but it expects students to be comfortable with the Git workflow and does not provide laboratory attendance or an integrated execution environment. Cloud IDEs offer a convenient programming environment in the browser but are not tied to institutional roles or evaluation.

Table 2.1: Typical capabilities of existing tool categories compared with LabFlow

Capability	LMS	Online judge	GitHub Classroom	Cloud IDE	LabFlow
Role-based portals (student / faculty / admin)	Yes	Limited	Limited	No	Yes
In-browser code editor and execution	No	Yes	No	Yes	Yes
Test-case based checking	Plugin	Yes	Via CI	No	Yes
Automatic push of work to student's GitHub	No	No	Yes	Manual	Yes
Laboratory timetable and attendance	Limited	No	No	No	Yes
Faculty marks and feedback per exercise	Yes	Limited	Limited	No	Yes
Timed exams with focus monitoring	Plugin	Yes	No	No	Yes
Institution audit log and maintenance mode	Yes	No	No	No	Yes

The table is a qualitative summary of what each category typically offers; individual products differ.

2.3 Technologies and Standards Reviewed

- REST architecture [5]: the back end exposes resources through stateless HTTP endpoints so that the React client and the API remain loosely coupled.
- OAuth 2.0 and OpenID Connect [3][17]: used for delegated sign-in with Google and for authorising LabFlow to write to a student's GitHub account without ever seeing the student's GitHub password.
- JSON Web Tokens [4]: compact signed tokens carry the user identity and role between the client and the API.
- PBKDF2 [6]: a standard password-based key derivation function used to hash passwords for the development and administrator accounts.
- Monaco Editor [11]: a browser-based editor with syntax highlighting, bracket matching and code suggestions that gives the IDE a familiar Visual Studio Code feel.
- Judge0 [12]: an open-source online code execution system that compiles and runs untrusted programs in a sandbox; LabFlow uses it as the fallback executor.
- GitHub REST API [13]: the contents endpoint is used to create or update a file in a repository as a commit.

- Fernet [14]: authenticated symmetric encryption, used to protect GitHub access and refresh tokens stored in the database.

2.4 Summary

The literature and existing tools show that the individual pieces of a programming laboratory workflow, namely editors, automatic testing, version control and learning management, are well developed, but they are usually offered as separate products. LabFlow combines them into one institution-specific platform. Its in-browser IDE removes the set-up barrier for students, and its automatic GitHub synchronisation turns every laboratory submission into a version-controlled record in the student's own repository.

Chapter 3
Implementation

This chapter describes how LabFlow is built: the overall architecture, the database design, the functional modules, and the main algorithms that implement the in-browser IDE, GitHub synchronisation, attendance and online exams.

3.1 System Architecture

LabFlow follows a three-tier client–server architecture. The presentation tier is a React single-page application served to the browser; it contains the student, faculty and administration portals and the Monaco-based IDE. The application tier is a FastAPI service that exposes a versioned REST API under /api/v1. It is organised into routers (authentication, student, faculty, admin, exams, health) and a service layer that holds the business logic. The data tier is MongoDB. Two external integrations complete the system: Google Identity for sign-in and the GitHub REST API for repository synchronisation, and a code execution engine is used by the service layer to run student programs.

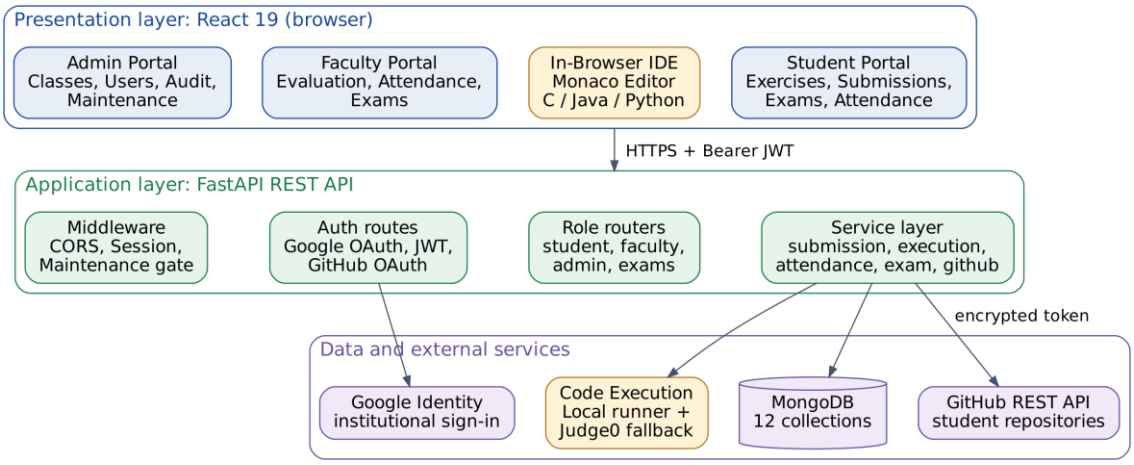


Figure 3.1: System architecture of LabFlow

Every request from the browser carries a Bearer JSON Web Token. A dependency in the API decodes the token, loads the corresponding student, faculty or administrator record and attaches the role, and each router accepts only the roles that are allowed to use it. On the client side a ProtectedRoute component applies the same role check before a page is rendered. A maintenance middleware in the API can place the whole system into maintenance mode, in which case all routes except the administration, authentication and health routes return a 503 response with the message and expected return time set by the administrator.

Presentation layer

The presentation layer is the React single-page application delivered through a web browser. Shared navigation, authentication state, form controls, notifications and API-client logic are reused across the student, faculty and administration portals. Students see laboratories, exercises, submissions, attendance, exams and the IDE. Faculty screens expose allocated laboratories and controls for exercises, test cases, submissions, attendance and exams. Administration screens focus on accounts, enrolments, announcements, audit records and system settings.

Route protection aligns each portal with the signed-in role. After login, the client requests the current profile and role, then opens the corresponding workspace. API calls include the bearer token and present consistent loading, success and error states. Monaco provides language modes, editing controls, standard input, execution output and error tabs. The same editor can be opened from a practice sandbox, an assigned exercise or an eligible exam question.

The browser is responsible for interaction and presentation, not authoritative decisions about grades, permissions or code safety. It may hide an unavailable action for usability, but the API repeats permission checks before changing data. Drafts are kept locally by language to reduce accidental loss, while submissions and evaluations are stored on the server.

Application layer

The application layer is a FastAPI service exposing a versioned REST interface. Routers separate authentication, student, faculty, administration, exams and health operations. Requests are parsed and validated before service logic runs. An authentication dependency verifies the signed JSON Web Token, resolves the profile and supplies the role. Resource checks confirm that a student owns an attempt or submission, or that a faculty member is allocated to the laboratory. These checks apply to direct API calls as well as browser requests.

Service functions coordinate multi-step actions. Exercise submission validates the exercise and language, creates or updates the student-exercise record, records status and timestamps, then attempts GitHub synchronisation when configured. The local submission remains the primary record if the remote push fails. Code execution validates the language and user, prevents overlapping executions for one student, selects an executor, enforces the time limit and normalises output.

Middleware and shared dependencies provide maintenance mode, CORS configuration, request identity and database initialization. A health endpoint supports operational checks. Centralising these rules avoids duplicating security and workflow logic in every page.

Data layer and external integrations

MongoDB collections represent the principal entities in the laboratory workflow. Profile collections keep role-specific identity data; laboratories connect faculty and cohorts to exercises; exercises may contain embedded test cases; submissions preserve code, evaluation and GitHub metadata; attendance records preserve session outcomes; and exams contain questions while exam attempts retain answers, deadlines and scores. Indexes support identity lookups and common review queries, and unique indexes protect records that must not be duplicated.

Google Identity participates in institutional sign-in. GitHub OAuth is separate: a student explicitly connects an account and protected tokens are encrypted before storage. The GitHub REST API can create a private laboratory repository and commit submitted source. Judge0 provides an external execution fallback when local toolchains are unavailable. These services are isolated behind integration logic so the core workflow can report failures in a controlled way.

The context and data-flow diagrams show the system boundary and how requests move between users, application processes, data stores and external services.

Security, trust boundaries and failure handling

All protected requests pass through the API authentication dependency. The server validates the token signature and expiry, resolves the profile and applies the role and resource checks required by the requested operation. Student access is scoped to the student's own work and enrolments; faculty access is scoped to allocated laboratories; administrator routes are restricted to authorised administrative profiles. The React client supports this model with protected routes, but the API remains authoritative so direct requests receive the same checks.

Code execution is treated as a separate service boundary because source code is supplied by users. The API accepts only the supported language identifiers, limits concurrent execution per student, applies a timeout and caps returned output. Compiler and runtime failures are returned as structured statuses that the IDE can display. Judge0 offers an isolated fallback; stronger container-level resource and network restrictions remain an identified future improvement for production hardening.

Integrations also have explicit failure behavior. Google sign-in failures return the user to a recoverable authentication state. GitHub credentials are encrypted, refresh is attempted when needed, and remote synchronisation status is stored separately from the local submission. Database connection and index initialization occur during application startup, while the health route helps administrators distinguish service availability from a browser-side issue. This design keeps errors visible without silently discarding student work.

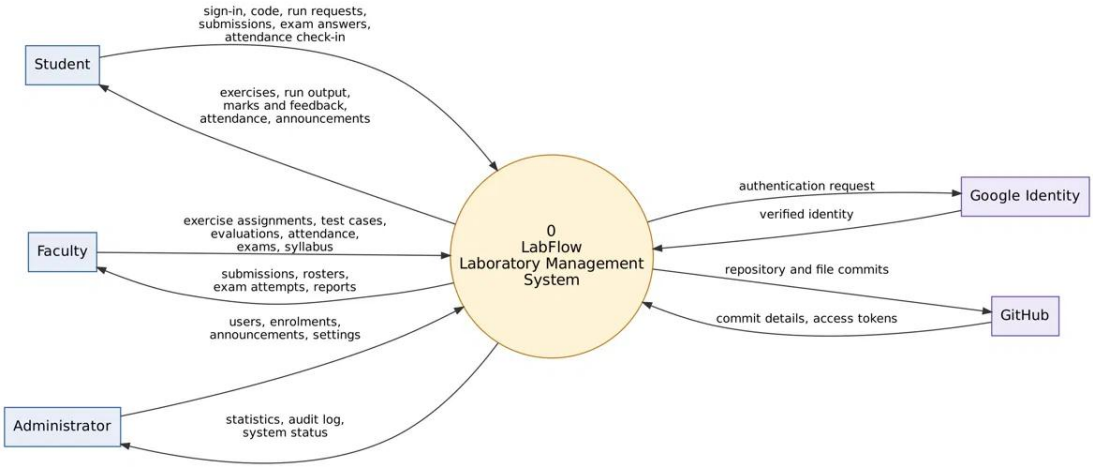


Figure 3.2: LabFlow context diagram and external information flows

Students send sign-in requests, code, submissions, exam answers and attendance check-ins; they receive exercises, execution output, marks, attendance summaries and notices. Faculty provide assignments, test cases, evaluations, attendance updates and exam definitions, and receive rosters and review information. Administrators manage accounts and settings and receive statistics and audit information. Google Identity and GitHub remain external trust boundaries: LabFlow requests identity verification or repository operations and processes the responses.

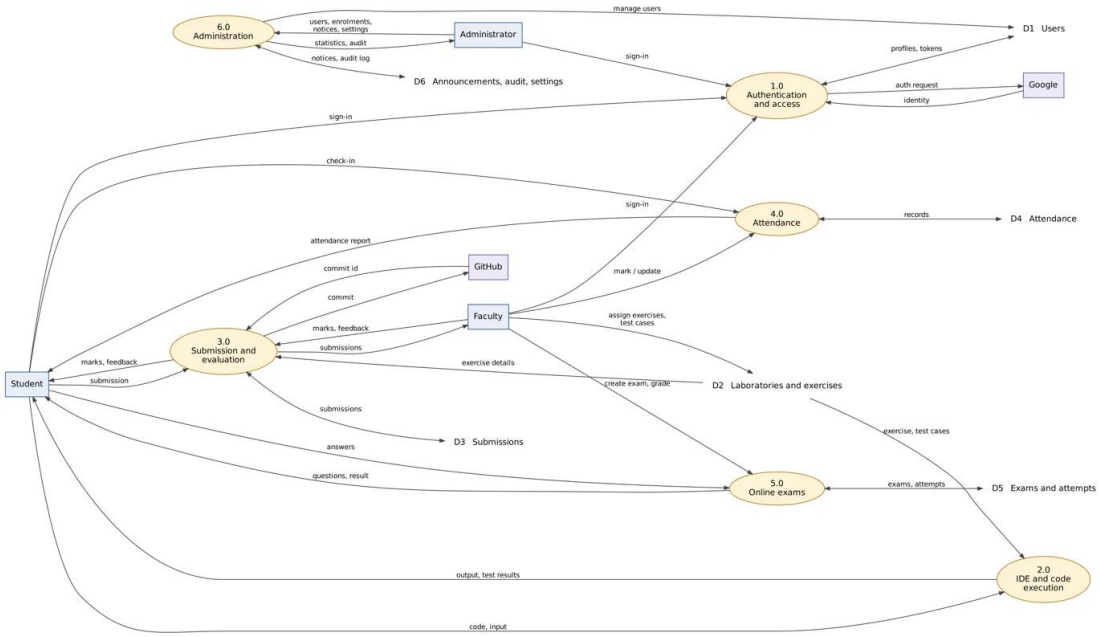


Figure 3.3: Level-one data-flow diagram for LabFlow

The level-one view separates authentication, IDE execution, submission and evaluation, attendance, exams and administration. These processes exchange information with collections for users, laboratories and exercises, submissions, attendance, exam attempts, announcements, audit records and settings. An IDE run reads the exercise context and returns output; a submission is stored as an evaluated record and may initiate a GitHub commit. Faculty grading updates the submission and returns marks and feedback.

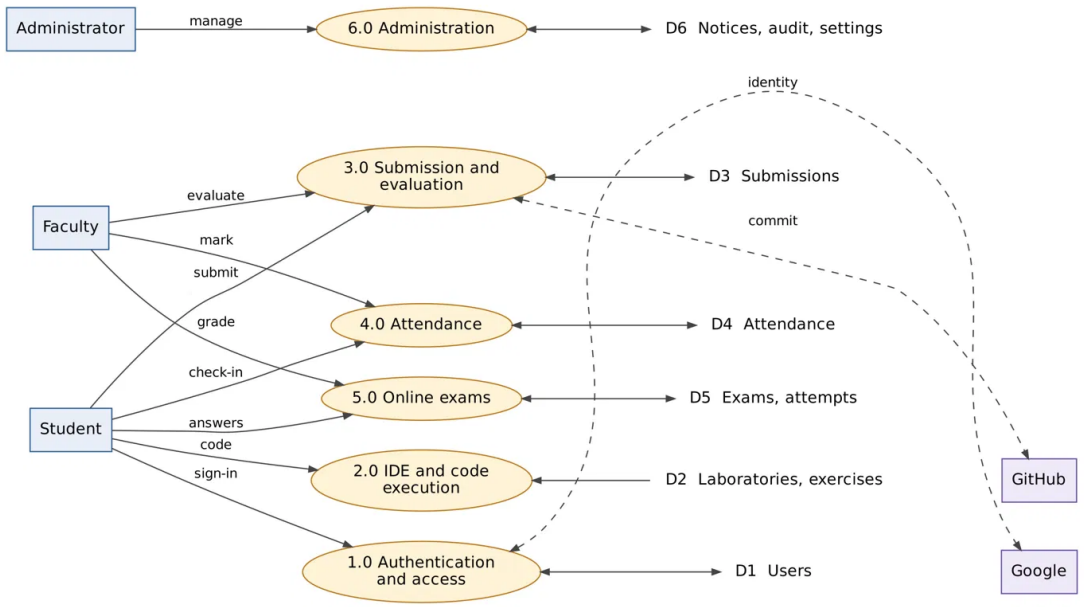


Figure 3.4: Data-flow view of LabFlow processes, integrations and data stores

This second data-flow view makes the persistent data stores explicit. User profiles are read and maintained during authentication; laboratory and exercise records provide the context for code work; submissions and exam attempts preserve assessed work; attendance records support session reporting; and announcements, audit records and settings support administration. The external Google and GitHub services exchange identity and repository information with the application but do not replace its own user and submission records.

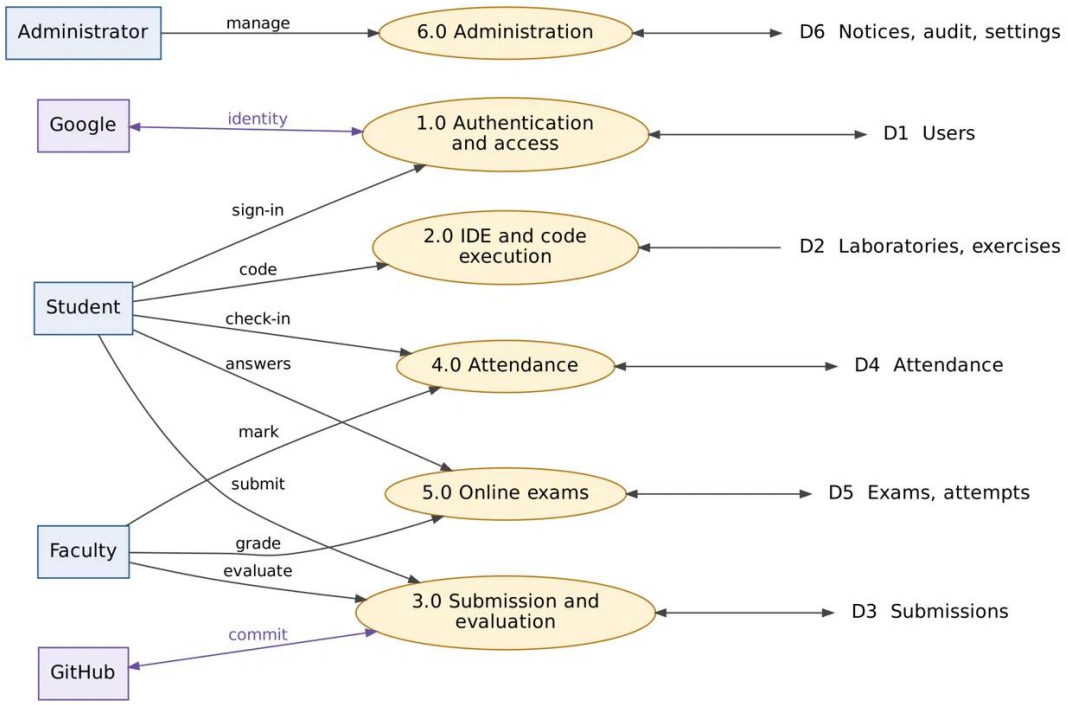


Figure 3.5: Data-flow diagram for role-based process interactions

The process-interaction view highlights how students, faculty and administrators initiate separate activities while sharing the same authenticated application. Students submit code and receive results; faculty evaluate work and maintain laboratory records; administrators manage accounts and system settings. Google Identity and GitHub exchange data through controlled integration points, while the central application remains responsible for access rules and persistent academic records.

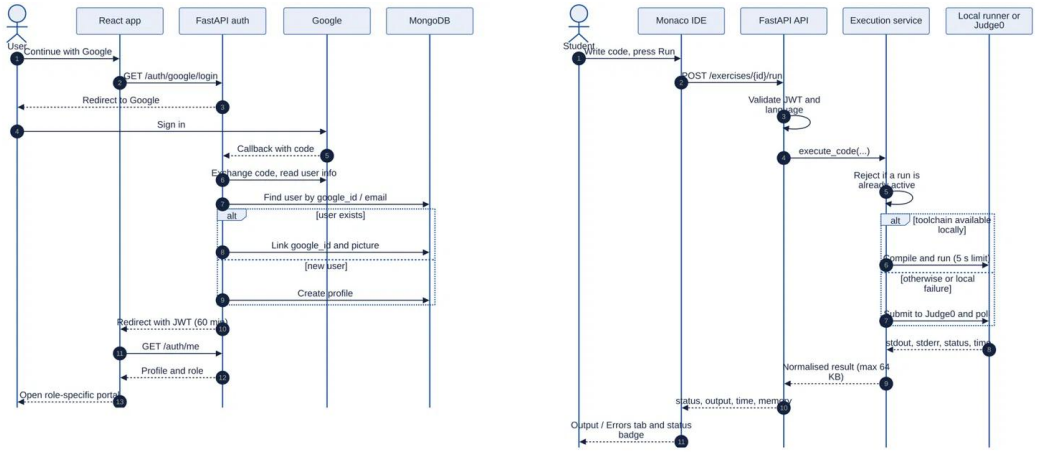


Figure 3.6: Authentication and IDE execution sequence

The sequence view illustrates two representative interactions. During Google sign-in, the client begins the flow, the API exchanges the callback with Google, resolves or creates the profile in MongoDB and returns a signed token. During a code run, the editor sends source and input to the student API; the service validates the request and chooses the local runtime or Judge0. A normalised result returns to the editor. An unavailable toolchain is handled as a controlled response rather than an unhandled browser failure.

3.1.1 Methodology

LabFlow was developed incrementally using the Scrum approach described in Section 1.6. Work was divided into reviewable increments so the core laboratory workflow could be checked before optional integrations were added. The first increment established the React client, FastAPI service, MongoDB connection and role-aware authentication. Later increments added student and faculty portals, exercise management, the browser IDE, code execution, submissions, attendance, GitHub synchronisation, online exams and administration features. Each increment was checked against the relevant user flow before the next capability was introduced.

The development process combined requirement analysis, interface design, implementation, integration and verification. Student, faculty and administrator tasks were mapped to portal screens and API operations. Data structures were then defined for users, laboratories, exercises, submissions, attendance and exams. Client components were connected to REST endpoints in small groups, making it easier to isolate errors in authentication, validation and role permissions. Integration work was followed by browser checks and automated API scripts; findings informed fixes to validation, execution limits, status handling and response messages.

Separation of concerns guided implementation. React handles presentation and client interaction; FastAPI routers validate requests and apply access rules; service functions implement workflows; MongoDB stores persistent records. Google Identity, GitHub and Judge0 are accessed through dedicated integration logic. A failure in an optional integration, such as GitHub synchronisation, is reported without losing the core submission record.

3.1.2 Hardware Requirements

The values below are practical recommendations for a small academic deployment, not measurements of the project development machine. A student or faculty client needs a current browser, stable network access and enough memory to keep the IDE and course

pages responsive. A dual-core computer with 4 GB RAM is a reasonable minimum; 8 GB RAM or more is recommended when several browser tabs or development tools are open.

A small server deployment can start with 2 virtual CPU cores and 4 GB RAM for the FastAPI service and front-end assets, provided MongoDB and code execution are sized separately or managed services are used. Larger classes, concurrent execution requests or locally hosted compilers require more CPU and memory. Storage should allow for logs, backups, submissions and uploaded syllabus files. Regular backups and disk monitoring are recommended. Google sign-in, GitHub operations and optional Judge0 execution require network access.

3.1.3 Software Requirements

Layer	Software / service	Purpose
Client	Current Chromium, Firefox or Edge; React 19; Router 7; Vite 8	Runs role portals and Monaco editor.
API	Python 3; FastAPI; Uvicorn; Pydantic; HTTPX; Authlib	REST endpoints, validation and identity callbacks.
Data	MongoDB with Motor / PyMongo	Persistent laboratory and user records.
Execution	Python runtime, GCC, OpenJDK; optional Judge0	Runs C, Java and Python with bounded time/output.
Integrations	Google Identity, GitHub REST API	Sign-in and repository synchronisation.
Development	Node.js/npm and Python test scripts	Builds client and verifies API workflows.

3.2 Database Design

LabFlow uses MongoDB, a document database, because the data of a laboratory system (exercises with embedded test cases, exams with embedded questions, submissions with embedded GitHub metadata) maps naturally to nested documents. The database contains twelve collections, described in Table 3.1.

Table 3.1: MongoDB collections

Collection	Purpose and main fields
students	Student profile: google_id, email, student_id (roll number), name, department, semester, profile_picture and a github object (connected, username, encrypted access and refresh tokens, expiry times).
faculty	Faculty profile: faculty_id, name, email, google_id, department and the laboratories allocated to the faculty member.
admins	Administrator accounts with email and hashed password.
laboratories	Laboratory courses: course id, code (for example 20MCA131), name, faculty, syllabus file and cohort.
exercises	Laboratory exercises: exercise_id, course_id, exercise_number, title, language, due date, is_assigned flag and an embedded list of test_cases (id, input, expected_output, is_hidden).
submissions	One document per student per exercise: submission_id, student details, course and exercise details, submitted_code, language, comments, status, marks, feedback, evaluated_by, timestamps and a github object (synced, repository, branch, path, commit_sha, commit_url).
attendance	One record per student, laboratory and date: status (Present, Late or Absent), time, source (automatic lab entry, manual check-in or faculty), marked_by and verification.
exams	Timed assessments: title, course, duration in minutes, published flag and an embedded list of questions (multiple choice or coding, with points and language).
exam_attempts	A student's attempt at an exam: attempt_number, status, answers, started_at, deadline_at, violation_count, question_scores, score and grading_status.
announcements	Notices published by administrators or faculty with audience, title, message and publisher.
audit_logs	Record of important actions: action, target, actor and a readable summary.
system_settings	Global settings such as maintenance mode, maintenance message, expected return time and the late-attendance grace period.

Indexes are created automatically when the API starts. Unique sparse indexes protect google_id, email and student_id on the students collection and the corresponding fields on faculty, admins and submissions. Compound indexes on (student_id, exercise_id) and (course_id, status) speed up the faculty review screens, and a unique index on (exam_id, student_id, attempt_number) guarantees that each numbered exam attempt exists only once.

The collections follow the way users perform laboratory work. A laboratory stores course identity, semester, faculty allocation and syllabus. Exercises refer to a laboratory and carry assignment state, due date, language and test cases. A submission refers to a student and exercise and contains source code, evaluation state, marks, feedback and optional GitHub commit details. Attendance is stored by student, laboratory and session date to prevent duplicate check-ins while allowing faculty correction.

Exam definitions hold duration, publication state and questions. A separate exam-attempt record stores the student, attempt number, deadline, answers, focus-violation count, question scores and grading state. This separation preserves student responses while faculty update the published assessment. Announcements, audit logs and system settings are stored separately because their access patterns differ from submissions and exams.

The schema diagram is a logical view. MongoDB stores documents rather than relational rows, so the references indicate identifiers and workflow links; test cases and exam questions may be embedded in parent documents. The diagram communicates ownership and cardinality without implying that MongoDB enforces foreign-key constraints.

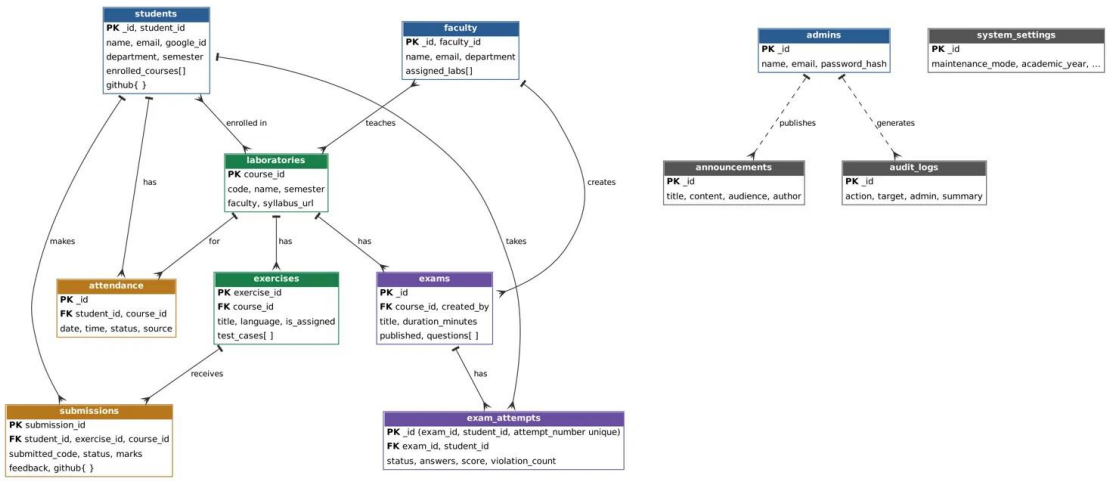


Figure 3.7: Logical MongoDB collection relationship diagram

Database access occurs through the service layer, not directly from the browser. Validation and role checks happen before persistent changes. Indexes support authentication lookup and faculty queries, while unique constraints prevent duplicate identities and repeated numbered exam attempts. A production operator should back up all collections and verify a restore in a separate environment.

3.3 Modules

Table 3.2: REST API endpoint groups

Router	Endpoints	Examples
Health	1	GET /health
Authentication	9	POST /auth/student/login, GET /auth/google/callback, POST /auth/github/login, GET /auth/me
Student	17	POST /student/exercises/{id}/run, /run-tests, /submit, GET /student/attendance, POST /student/sandbox/run
Faculty	22	PATCH /faculty/exercises/{id}/assign, PUT /faculty/submissions/{id}/evaluate, POST /faculty/attendance/{course}/batch
Exams	11	POST /exams/{id}/start, PUT /exams/{id}/answer, POST /exams/{id}/violations, PUT /faculty exam attempt grade
Admin	25	GET /admin/stats, POST /admin/faculty, PUT /admin/enrollments/{id}, POST /admin/announcements, POST /admin/maintenance

3.3.1 AUTHENTICATION AND ACCESS CONTROL

Students and faculty sign in with their institutional Google account. The back end uses Authlib to run the OAuth 2.0 / OpenID Connect flow. When Google returns the user's identity, LabFlow looks the user up by Google identifier, then by e-mail address, links the Google account to an existing profile if one is found, and otherwise creates a new profile. A JSON Web Token containing the user id, role, name, e-mail and picture, valid for 60 minutes, is issued and the browser is redirected to the front end, which stores the token and loads the correct portal. For development and for administrators, e-mail and password login is also available; passwords are never stored in plain text but as PBKDF2-HMAC-SHA256 hashes with a random 16-byte salt and 100,000 iterations, compared in constant time.

The callback exchanges the authorisation code, reads verified identity claims and links the Google identifier to a matching profile or creates a new account. The server issues a short-lived signed token containing the role and user identifier. Protected routes resolve that identity for every request; browser route protection is only a usability layer. Administrator password credentials use salted PBKDF2 hashes, while GitHub tokens are encrypted separately and excluded from client profile responses.

After sign-in, the API checks the token signature and expiry before processing a protected request. It also checks the requested record against the caller's role and ownership: students can access their own submissions and attendance, while faculty actions are limited to allocated laboratories. This keeps access rules in the server even if someone bypasses a portal screen or changes a request.

The same identity is used across the student, faculty and administration portals, but each role receives only the functions needed for its work. Password-based administrator access is stored as a salted PBKDF2 hash, and GitHub credentials remain separate from sign-in credentials. These controls reduce the chance that a copied profile response or an expired session exposes an integration secret.

3.3.2 STUDENT PORTAL

After login a student sees a dashboard with the assigned laboratories, overall attendance, upcoming laboratory sessions and recent activity. The My Laboratories page opens a laboratory with its faculty, syllabus and exercise list; the Exercises page lists every exercise across all laboratories with its due date and submission status (Not Submitted, Submitted, Evaluated); the Submissions page shows past submissions with marks and faculty feedback; and the Profile page lets the student update personal details and connect GitHub. A notification popover shows announcements published by the administration and faculty.

The student portal follows the weekly lab cycle: dashboard, course materials, assigned exercises, coding, submission, feedback and attendance. Status labels distinguish work not submitted, awaiting review and evaluated. The submission page reads the authoritative server record, while the IDE may keep a local draft to reduce accidental loss. Course and ownership checks restrict the records returned to each student.

Course and exercise pages help students move from a laboratory's syllabus to the work assigned for that course. Due dates and submission states provide a quick progress check, while links to the IDE and exercise details keep the instructions close to the coding task. The dashboard therefore acts as the starting point for routine laboratory work.

After submitting, students can return to the submission history to compare the current status with earlier work and read faculty comments. Attendance summaries and audience-specific announcements give students the information needed to plan upcoming sessions and respond to course updates. The portal presents these records from the student's enrolled courses and personal account.

3.3.3 IN-BROWSER IDE AND CODE EXECUTION ENGINE

The IDE is the central feature of LabFlow. It is available in two forms. The exercise IDE opens from an assigned exercise and shows the exercise description beside the editor; the standalone IDE (the sandbox) is a free playground for practice. Both use the Monaco editor, configured with the dark theme, line numbers, bracket-pair colouring, automatic closing of brackets and quotes, code suggestions and word wrap. The student chooses C,

Java or Python, writes the program and presses Run. A Standard Input panel allows custom input to be supplied, and the result appears in the Output, Input and Errors tabs together with a status badge such as SUCCESS, Compilation Error, Runtime Error or Time Limit Exceeded, the execution time and memory use.

For convenience the IDE loads a starter template for the chosen language that greets the student by name, saves the draft for each language automatically in the browser so that work is not lost on refresh, and offers a one-click reset to the template. In the exercise IDE, Run Tests executes the code against the faculty-defined test cases; the results for visible test cases show actual and expected output, whereas hidden test cases report only pass or fail so that students cannot hard-code answers. Submit stores the solution for faculty review.

On the server, the execution service validates the language, allows only one active execution per student, and then selects the executor. Python, Java and C programs are run by a local runner that writes the code into a temporary directory, compiles it when necessary (gcc for C with the math library, javac for Java), and runs it with the student's input under a configurable time limit (five seconds by default). If a toolchain is not available locally, or the local run fails, the request is forwarded to a Judge0 sandbox, which is polled until the result is ready. Whichever executor is used, the result is normalised into one structure containing status, standard output, standard error, compiler output, exit code, execution time and memory, and the output is truncated at 64 KB. During an exam, code execution is allowed only for the matching coding question of an active attempt.

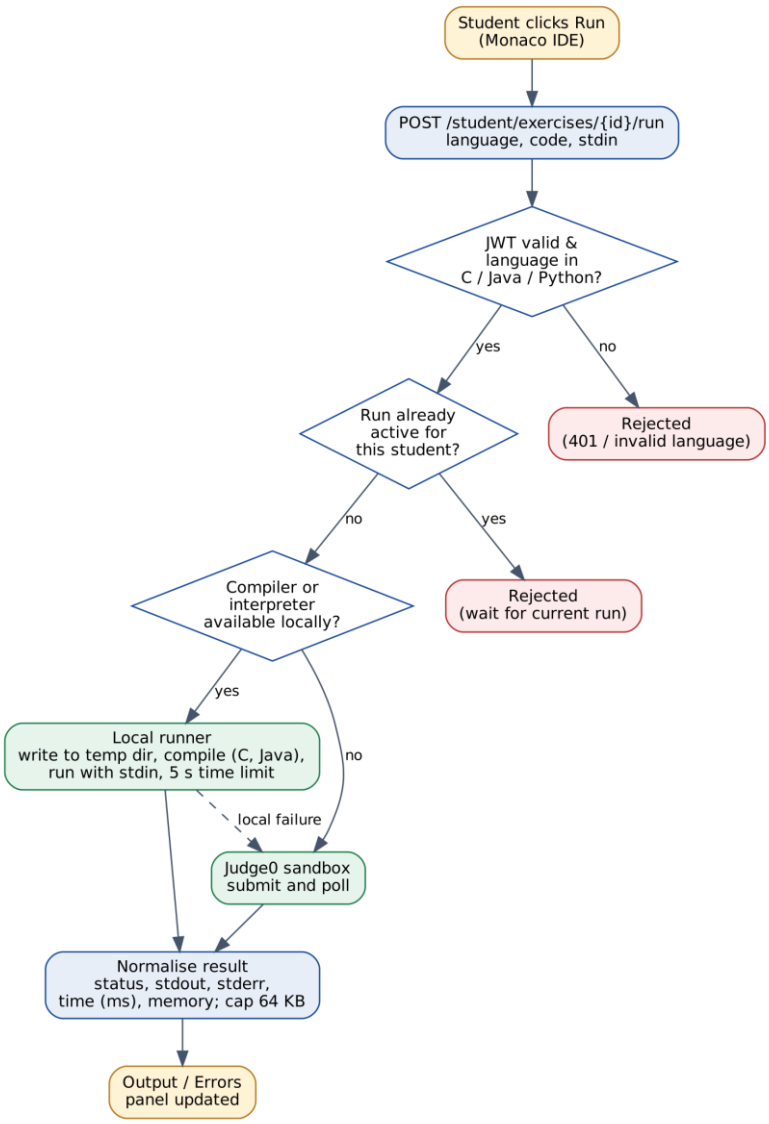


Figure 3.8: Flow of a Run request through the code execution engine

The IDE separates editing, execution, testing and submission. Run sends source and optional input and returns status, output, diagnostics, exit code and timing. Run Tests executes visible and hidden cases without submitting; hidden cases reveal only pass/fail. Submit persists work for faculty review. During an exam, the API restricts execution to the active coding question. A per-student run guard, time limit and output cap bound execution; Judge0 is available when local toolchains are missing.

The editor gives students a repeatable edit–run–inspect cycle. They can provide standard input, read compiler messages and runtime output, correct the source and run it again before submitting. Separating a practice run from a formal submission lets students experiment without changing the submission record that faculty will evaluate.

Execution is bounded to protect the application from programs that run indefinitely or produce excessive output. The service returns a consistent result containing status and diagnostics, so the browser can present failures in the same way across supported languages. Exam requests are additionally checked against the active attempt and coding question.

3.3.4 GITHUB INTEGRATION AND SYNCHRONISATION

The second major feature is automatic GitHub synchronisation. On the Profile page a student presses Connect GitHub, which starts a GitHub OAuth authorisation. After the student approves, LabFlow receives access and refresh tokens, encrypts them with Fernet and stores them in the student's record; the tokens are removed from every profile response and are never sent to the browser. A Disconnect button removes the connection at any time.

Once connected, every submission is saved directly to GitHub in addition to the LabFlow database. LabFlow derives a repository name from the laboratory name, checks whether the repository exists in the student's account, and creates a private repository with an initial commit if it does not. It then commits the solution as a file named after the exercise (for example exp1.c, exp2.py or exp3.java) with a descriptive commit message such as “LabFlow: Submit 20MCA131 Directory Tree & Linux File Operations”. Re-submitting the same exercise updates the file and creates a new commit, so the repository keeps the full history of the student's work. The access token is refreshed automatically when it has expired, and the push is retried with increasing delays to cope with the short period after a repository is created before GitHub is ready to accept files. The result of the synchronisation (repository, path, commit id and link, or the error) is stored with the submission and shown in the IDE as the GitHub Sync status. A synchronisation problem never prevents the submission itself from being saved.

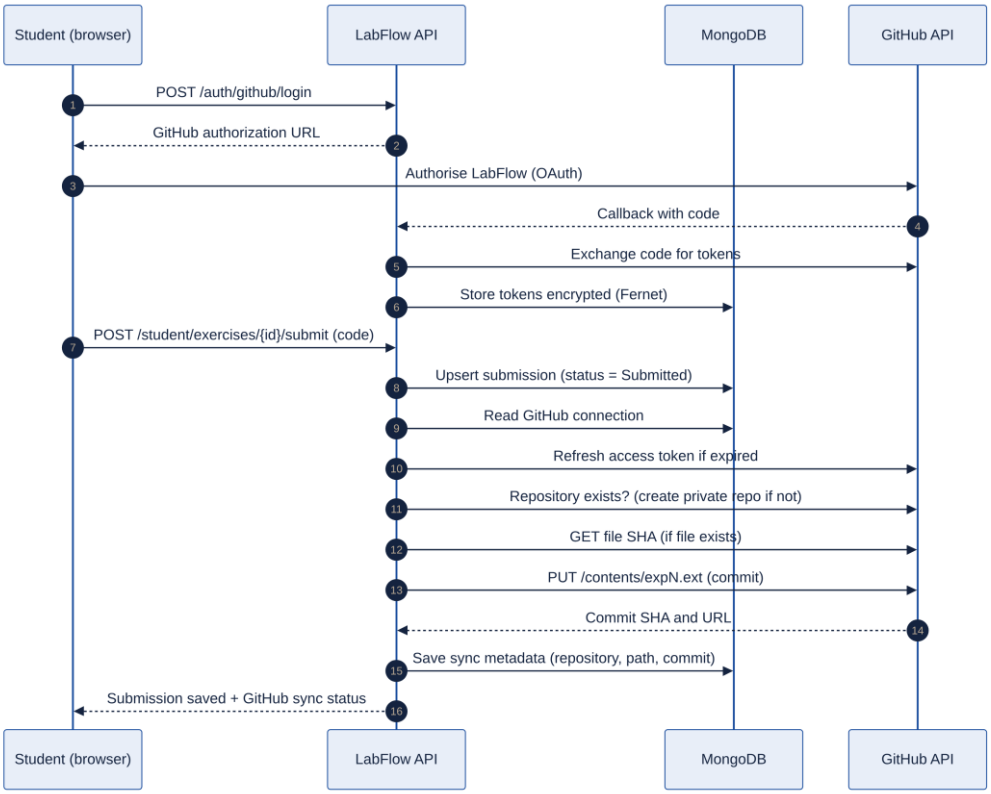


Figure 3.9: Sequence of GitHub connection and submission synchronisation

GitHub connection begins only after the student selects Connect GitHub and approves OAuth. Tokens are encrypted at rest and removed from public responses. On submission, LabFlow first stores the local academic record, then creates or resolves a private laboratory repository and writes a predictable exercise file through the GitHub REST API. Repository, branch, path, commit id and URL are recorded when available. Refresh and retry logic handles expired credentials and transient provider delays; a remote failure never rolls back the local submission.

The integration organises work by laboratory repository and exercise file, giving each saved solution a clear location in the student's GitHub account. A later submission updates that exercise file and creates a new commit, which preserves earlier versions and makes progress visible through the repository history.

The local submission and remote synchronisation have separate outcomes. LabFlow first saves the academic record, then records the repository, branch, file path, commit identifier and link returned by GitHub. If the account is disconnected or GitHub is temporarily unavailable, the student still has a saved submission and can see that synchronisation needs attention.

3.3.5 SUBMISSION AND EVALUATION

When a student submits, the system checks that the exercise exists and has been assigned by faculty, validates the language, creates or updates the single submission document for that student and exercise, and sets its status to Submitted while preserving any earlier marks and feedback. Faculty open the Submissions tab of a laboratory, filter by exercise or status, read the submitted code, and record marks, feedback and the new status (for

example Evaluated). The action is restricted to faculty who are allocated to that laboratory and is written to the audit log. Students see the marks and feedback on their Submissions page.

The submission service validates assignment and language, then creates or updates the student-exercise record with source, status and timestamps. Faculty filter by course, exercise or review status, inspect code and record marks and feedback. The API checks that the reviewer is assigned to the laboratory, and important evaluation changes can be traced in the audit log. Students read the same evaluation record from their portal.

The submission workflow gives each student and exercise a record that can move from submitted to evaluated. Validation happens before the record is accepted, and the saved source and timestamps give faculty a stable version to review. The academic status remains independent of any GitHub synchronisation result.

Faculty can work through the review queue by course, exercise or status, open the submitted source and record a mark with explanatory feedback. The system records who performed the evaluation and when it was made. Students see the resulting mark and comments in their account, giving them a clear route from submission to review.

3.3.6 ATTENDANCE

Laboratory sessions are defined in a weekly timetable that records the day, start and end time, laboratory, faculty and location. When a student signs in or enters a laboratory while a scheduled session is running, attendance is recorded automatically as Present, or as Late if the student arrives after the configurable grace period (ten minutes by default). Students can also check in manually, only while a session is active. Faculty see an attendance roster for each session, can change the status of one student or mark the whole class in a batch, and the student's Attendance page shows the overall percentage, sessions attended, the next laboratory and a per-laboratory breakdown. Duplicate records for the same student, laboratory and day are prevented.

Attendance combines the scheduled session with the identity and role of the user. During an active session a student can be marked automatically or check in manually; a configurable grace period distinguishes Present from Late. Faculty may correct an individual record or mark a class in a batch. Each record keeps its source and marker so the system can explain how the status was produced. A uniqueness rule prevents duplicate records for the same student, lab and date.

A session record connects a student to a particular laboratory meeting and records the attendance status and source of the check-in. The timetable and grace period determine whether the student is marked Present or Late, and the roster gives faculty a view of the class for that session.

Faculty can make a correction for an individual student or update the roster for the class, with the action associated with its source and marker. The portal summarises the stored records for students and faculty, while preventing duplicate entries for the same student, laboratory and date.

3.3.7 ONLINE EXAMS

The Exams module lets faculty create a timed assessment for a course, with multiple-choice and coding questions, and publish it to the students of that course. A student starts an attempt, the system fixes a deadline from the time limit, and answers are saved as the student works. Multiple-choice questions are graded automatically when the attempt ends; coding questions are reviewed and scored by the faculty member in the review desk. A focus guard records a violation whenever the student leaves the exam window and terminates the attempt automatically when the number of violations exceeds the permitted limit. Attempts that pass their deadline are closed automatically, and retakes are controlled by a configuration switch.

Starting an exam creates an attempt with a fixed deadline. Answers are saved only while the attempt is active; the API rejects changes after expiry or closure. Multiple-choice questions can be scored automatically, while coding answers can remain pending for faculty review. The focus guard records window-leave violations and terminates the attempt when the configured threshold is exceeded. Question scores, overall result and grading status remain attached to the attempt.

An exam is published to students eligible for the selected laboratory and contains a defined duration and question set. When a student starts, LabFlow creates an attempt with its own start time and deadline; saved answers belong to that attempt, allowing the system to keep separate records for separate students and exam sittings.

At submission or expiry, the service closes the attempt and retains its answers, score and grading state. Multiple-choice results can be calculated automatically; coding responses remain available for faculty review. The focus guard records window changes and applies the configured violation threshold, keeping the outcome and any termination reason with the attempt.

3.3.8 FACULTY PORTAL

The faculty dashboard summarises the number of laboratories, enrolled students, submissions received and submissions pending review, and lists the students who need attention. Inside a laboratory the faculty member can view and update the syllabus, assign or withdraw exercises, define test cases for each exercise, review submissions, view the enrolled students and publish laboratory announcements. The faculty portal also contains the attendance roster and the exam studio.

The faculty workspace is scoped to allocated laboratories and combines exercise, submissions, students, syllabus, announcements, attendance and exam tools. Test-case controls distinguish visible from hidden cases; submission filters reduce review effort; batch attendance supports routine sessions. The exam studio creates and publishes timed assessments and routes coding responses to manual review. These operations use the same API role and resource checks as other faculty actions.

The portal groups teaching tasks by assigned laboratory, making it easier to move between course setup and day-to-day review. Faculty can maintain exercise instructions

and tests, check the enrolment list, and publish announcements to the relevant students. Submission filters make pending work visible alongside its current status.

Attendance and examination tools use the same course allocation as the rest of the portal. Faculty can record a session roster, prepare and publish a timed exam, then inspect attempts that require manual grading. The API checks the allocation for each operation, maintaining the same boundary for browser actions and direct requests.

3.3.9 ADMINISTRATION PORTAL

Administrators see system-wide statistics (students, faculty, laboratories, classes) and the academic programmes with their semester sections. They can add and edit students and faculty, change student status, reassign laboratory courses between faculty, manage enrolments, publish and delete institutional announcements for a chosen audience, review the audit log, edit system settings and switch maintenance mode on or off with a message and expected return time.

Administrators maintain users, faculty allocations, enrolments, announcements, audit records and operational settings. Maintenance mode is enforced by API middleware so it applies consistently to pages and direct API calls. The audit log records the actor, target and action summary. Account and allocation changes are reflected in the same MongoDB records used by the portals, keeping course and identity data consistent.

Administrative updates keep course and account information aligned for the current term. An administrator can correct profiles, update enrolments and faculty allocations, and use system-wide counts to review the set of active students, faculty, laboratories and classes.

Notices can be directed to the intended audience, while system settings control operational behaviour such as maintenance mode and attendance timing. Administrative actions are recorded with the actor, affected target and a summary, providing a history that can be reviewed when investigating a change or confirming who performed it.

3.3.10 UML Diagrams

The following diagrams complement the module descriptions. The use-case view maps role permissions, the class diagram summarises principal domain objects and their associations, and the activity diagram follows a student exercise through faculty evaluation. These are logical design views; implementation details such as MongoDB embedding and REST routing are explained in the surrounding sections.

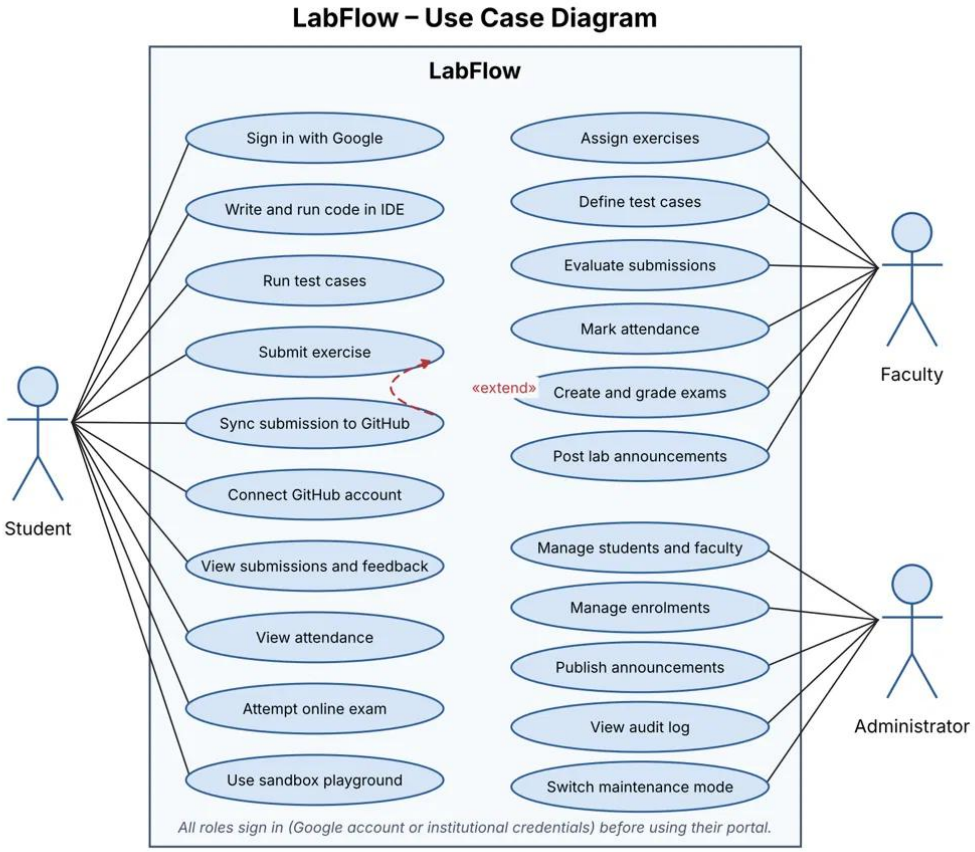


Figure 3.10: LabFlow use-case diagram for student, faculty and administrator roles

Students use the IDE, run tests, submit exercises, connect GitHub, review feedback, check attendance, attempt exams and use the sandbox. Faculty assign exercises, define test cases, evaluate work, mark attendance, manage exams and publish laboratory notices. Administrators maintain users, enrolments, announcements, audit logs and maintenance settings. GitHub synchronisation extends submission when a student has connected an account; it does not prevent the core submission from being saved.

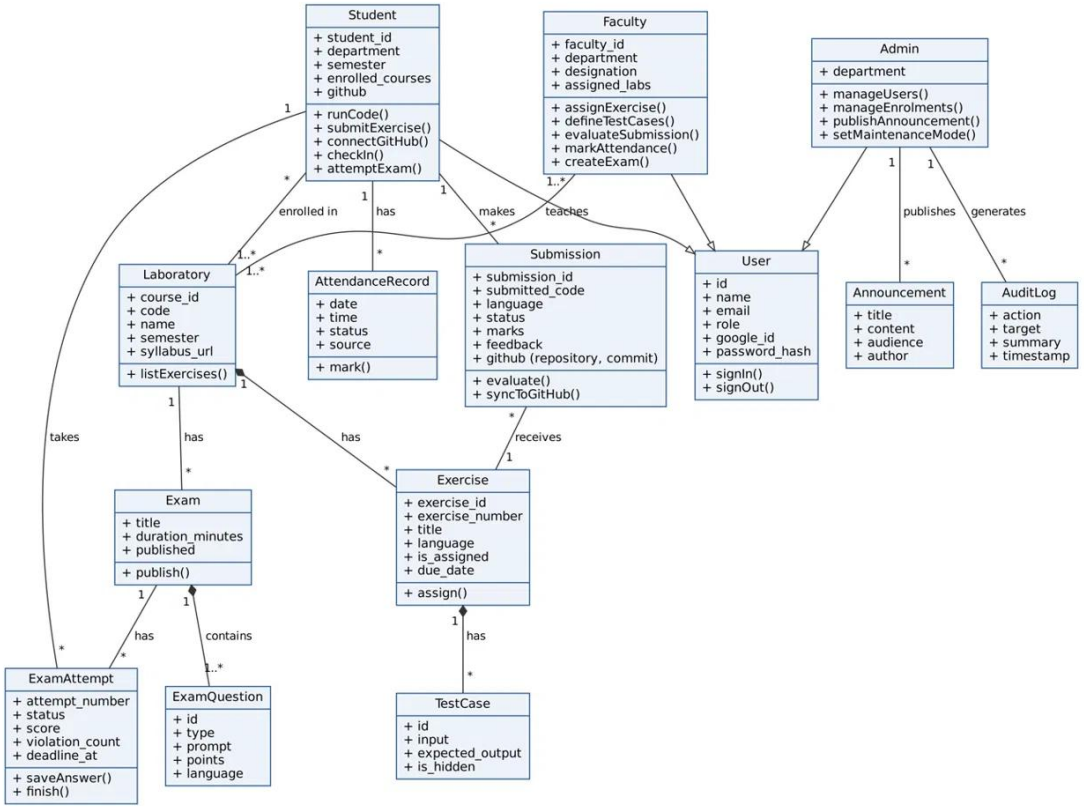


Figure 3.11: UML class diagram of the LabFlow domain model

The class diagram shows role-specific profiles associated with a common identity. Students enrol in laboratories and produce submissions, attendance records and exam attempts. A laboratory contains exercises and exams; an exercise has test cases and receives submissions; an exam contains questions and has separate attempts. Announcements and audit records are created by privileged users. MongoDB stores these concepts as collections and nested documents as described in Section 3.2.

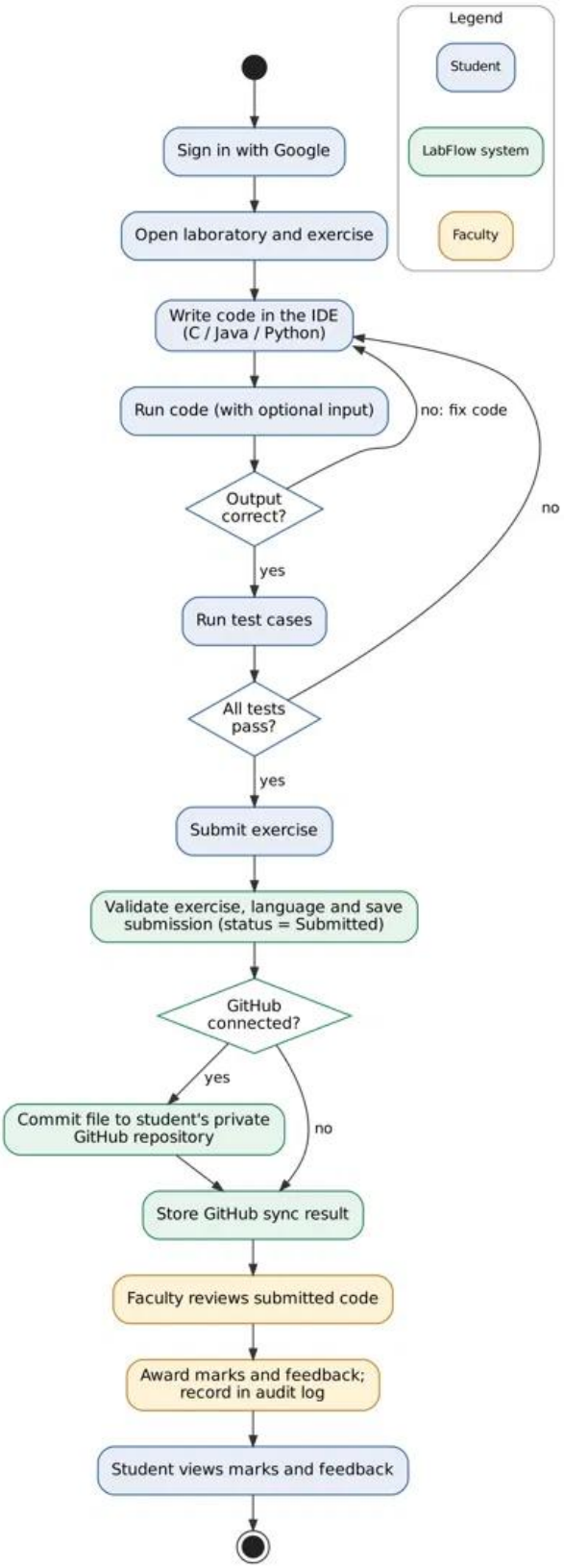


Figure 3.12: Activity diagram for the student exercise and review workflow

The activity view follows the feedback loop from sign-in and coding through test execution, submission, optional GitHub synchronisation, faculty review and student feedback. The student may correct code and repeat execution before submitting. The

local submission record remains the academic source of truth even when remote synchronisation fails.

3.3.11 Issues Faced and Remedies Taken

Issue encountered	Remedy implemented
Compiler/interpreter availability differs by host.	Use a common execution service, configured time limit and Judge0 fallback.
Untrusted code may run indefinitely or produce huge output.	Bound execution time, allow one active run per student and cap returned output.
GitHub outages or expired credentials could interrupt submission.	Store local submission first; encrypt tokens, refresh credentials and record sync status separately.
Network retries can repeat attendance or late exam actions.	Use uniqueness rules and active-attempt/deadline validation.
A user may call APIs outside the normal portal.	Resolve JWT role and check resource ownership on every protected API action.

These remedies are reflected in the service boundaries and stored status fields. They limit the effect of common workflow and integration failures while keeping clear whether an action completed locally, remotely or not at all.

3.4 Algorithms

The important procedures of the system are given below in pseudo-code.

Algorithm 1: Google sign-in and token issue

```
1. Receive callback from Google with authorisation code
2. userinfo <- exchange code for tokens and read identity (sub, email, name,
   picture)
3. user <- find user by google_id
4. if user is not found and email is present: user <- find user by email
5.     if found: link google_id and picture to the profile
6. if user is still not found: create a new profile from userinfo
7. if role is student and a lab session is active now: record attendance
   automatically
8. token <- sign JWT { sub, role, name, email, picture, exp = now + 60 min }
9. redirect browser to FRONTEND_URL/auth/callback?token=token
```

Algorithm 2: Executing student code

Input: student_id, language, code, stdin

1. if language not in {c, java, python}: return Unsupported language
2. if an execution is already running for student_id: return busy message
3. mark student_id as running
4. if language is available locally:
5. write code to a temporary directory
6. if language is c or java: compile; on failure return Compilation Error
7. run program with stdin and the time limit
8. if time limit is exceeded: return Time Limit Exceeded
9. if exit code is non-zero: return Runtime Error, else return Success
10. else (or if the local run fails): submit to Judge0 and poll until finished
11. normalise result; truncate stdout and stderr to 64 KB
12. unmark student_id and return result

Algorithm 3: Submit with GitHub synchronisation

Input: student, exercise_id, code, language

1. check that the exercise exists and is assigned; validate language
2. upsert submission {student, exercise} with status = Submitted
3. if student has connected GitHub:
4. token <- decrypt access token; refresh it first if it has expired
5. repo <- sanitise(laboratory name); path <- "exp" + exercise number + extension
6. if repository repo does not exist: create private repository; wait 2.5 s
7. repeat up to 3 times:
8. sha <- current file sha (none if file is absent)
9. PUT contents/path with message, content, branch, and sha if present
10. on success stop; on conflict wait (2 s x attempt) and retry
11. store synced, repository, path, commit_sha, commit_url (or error) in submission
12. return saved submission with GitHub status

Algorithm 4: Attendance marking

Input: student, course_id

1. if student is not enrolled in course_id: return not enrolled
2. if a record exists for (student, course_id, today): return already recorded
3. session <- scheduled session for course_id that is active now
4. if there is no active session: return no active session
5. if current time <= session start + grace period: status <- Present else Late
6. upsert attendance {student, course_id, date, time, status, source}
7. return status

Algorithm 5: Exam attempt, focus guard and grading

1. start: create attempt number n with deadline = now + time limit
2. save_answer: allowed only while status = in_progress and now < deadline
3. violation: increment violation_count atomically while in_progress
4. if violation_count > 2: finish attempt with status = terminated
5. finish: score <- sum of automatic scores of multiple-choice questions
6. grading_status <- pending if any written / coding question exists, else graded
7. store question_scores, score and submitted_at
8. faculty grade: set marks for coding questions and feedback; grading_status <- graded

Algorithm 6: Running test cases

Input: exercise, language, code

1. for each test case (input, expected_output, is_hidden) in order:
2. result <- execute code with stdin = input
3. if execution completed: pass if trimmed stdout equals trimmed expected_output
4. else fail
5. otherwise mark the test case as error with the execution status
6. reveal actual and expected output only when is_hidden is false
7. overall <- passed if all pass; partial if some pass; error if none pass and errors exist; else failed

Integration workflow: exams and code execution

When a student presses Run, Monaco sends the selected language, source and optional standard input to the student API. The API validates the token and request, then invokes the execution service. The service verifies the language and active-run state, chooses the local runner when available or submits to Judge0, and returns a normalised status and output.

An exam adds an attempt boundary to the same execution path. Faculty publish an exam with a time limit; the student starts an attempt and receives a fixed deadline. Answers are saved while active, code execution is restricted to the current coding question, and focus violations are recorded. At submission or expiry, objective questions are scored automatically and coding answers remain pending until faculty review.

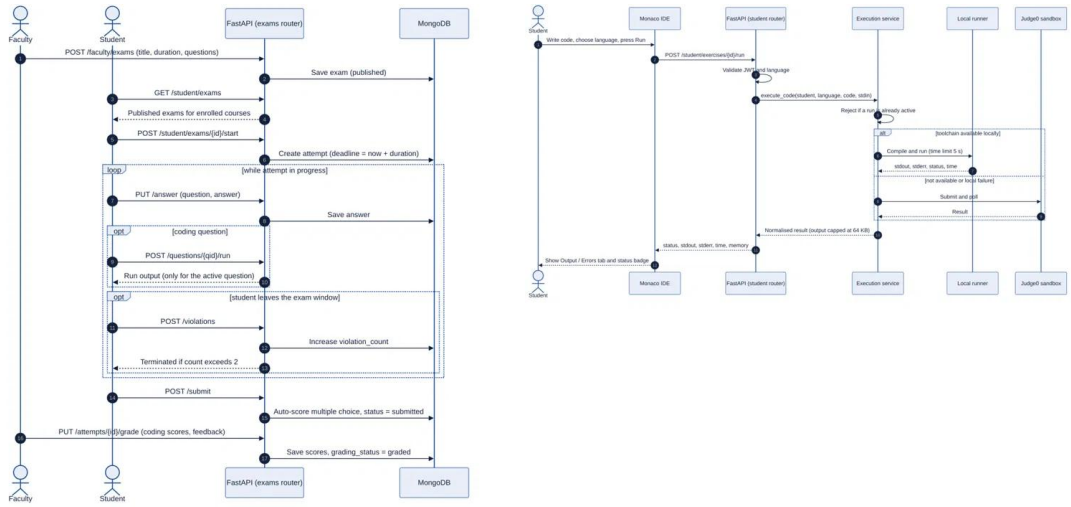


Figure 3.13: Sequence diagram for exam management and code execution

The sequence diagram combines faculty exam setup, a student attempt, the API, database and execution service. Alternative paths show local execution when toolchains are available and Judge0 polling as fallback. The exam portion shows answer persistence, focus violations, automatic scoring and faculty grading.

Chapter 4

Result Analysis

This chapter presents the outcome of the work. The results are based on the behaviour of the running system, as captured in the screenshots of Chapter 7, and on measurements of the code execution engine taken for this report.

4.1 Testing, Test Cases and Test Results

Testing combined API-level workflow scripts with manual verification through the student, faculty and administration portals. The cases below summarise outcomes reported for the running system and its repository verification flows. They cover normal use, validation failures, execution boundaries and integrations; they are functional checks rather than a claim of exhaustive security or load testing.

ID	Test case	Expected result	Observed result
01	Google sign-in / role route	Verified user opens correct portal.	Role-specific portal check passed.
02	Run C, Java and Python	Supported code returns output.	All three returned output; see Table 4.3.
03	Compile / runtime errors	Return correct status and diagnostics.	Both error types were classified.
04	Infinite loop	Stop at the time limit.	Timed out at about five seconds.
05	Visible / hidden tests	Show visible details; hidden pass/fail only.	Visibility rules applied.
06	Submit and grade	Store code, marks and feedback.	Submission and review flows covered.
07	GitHub unavailable	Keep local work and report sync separately.	Local-first; sync result separate.
08	Attendance retry / exam deadline	Prevent duplicates and reject late updates.	Uniqueness and deadline checks applied.
09	Faculty / admin roles	Restrict unpermitted operations.	Role and audit flows covered.

4.2 Functional Results

Table 4.1: Functional outcomes of the implemented modules

Module	Observed outcome	Evidence
Login	Institutional Google sign-in, a quick student login for development, and separate Faculty and Administrator logins are presented on one page.	Figure 7.1
Student dashboard	Greeting, programme and semester (MCA S3 · Computer Applications), overall attendance 81.8% and cards for the three laboratories with exercise counts and faculty names.	Figure 7.2
Laboratory and exercises	Laboratory page lists exercises with due dates and status; the Exercises page lists the assigned exercises across NSA, ADBMS and Java with a course filter.	Figures 7.3, 7.4
In-browser IDE	A Python program that sums a list ran in the browser and displayed 150 with the status SUCCESS.	Figure 7.5
Attendance	9 of 11 sessions attended, 81.8% overall; NSA 5/5 (100%), ADBMS 2/4 (50%), OOP Lab 2/2 (100%); next laboratory shown.	Figure 7.6
Faculty dashboard	Summary of laboratories, students, submissions received and 16 submissions pending review, with a Needs Attention list and class attendance average of 93.4%.	Figure 7.7
Faculty laboratory view	Tabs for Overview, Exercises, Submissions, Students and Announcements, syllabus update and laboratory operation counts.	Figure 7.8
Exam studio	Faculty can create and publish a timed assessment with coding and multiple-choice questions and review submissions.	Figure 7.9
Administration	Announcements published to the whole institution, and an overview of students, faculty, laboratories, classes and programme sections.	Figures 7.10, 7.11

4.3 Code Execution Performance

The local execution engine of LabFlow was measured with small test programs for each language, including programs with deliberate errors. Each case was run three times and the mean execution time is reported in Table 4.3 and Figure 4.1. The measurements were taken in a cloud Linux container, so absolute values on the institution's server will differ.

Table 4.3: Measured behaviour of the code execution engine

Test program	Result reported by LabFlow	Output	Time (ms)
Python: sum of a list	Success	150	21
Python: reads n from stdin and prints 1 + … + n (n = 100)	Success	5050	17
Python: division by zero	Runtime Error	(traceback in Errors tab)	31
Python: infinite loop	Time Limit Exceeded	Execution timed out	about 5,000 (limit)
C: sum of 1 to 10	Success	55	71
C: missing semicolon	Compilation Error	(compiler message)	8
Java: sum of 1 to 10	Success	55	732
Java: type mismatch	Compilation Error	(compiler message)	634

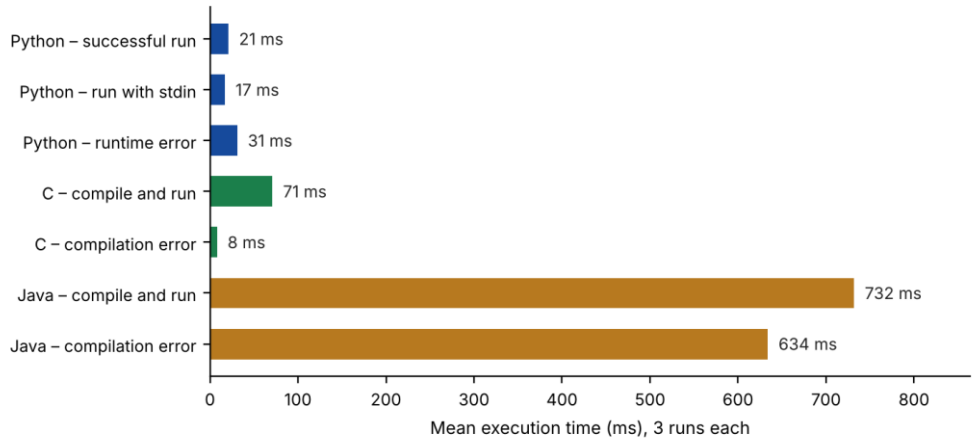


Figure 4.1: Mean execution time of the local execution engine

The results show that all three languages return the correct output and that the engine classifies each outcome correctly: successful runs, compilation errors, runtime errors and non-terminating programs. Python and C results are returned in well under a tenth of a second, which makes the IDE feel interactive. Java takes longer because the Java virtual machine must start for every run, but a result is still returned in under one second. The infinite loop is stopped at the configured five-second limit, so a faulty program cannot block the server.

4.4 Comparison with the Manual Laboratory Process

Table 4.4: Manual laboratory process compared with LabFlow

Activity	Manual process	With LabFlow
Writing and running code	Needs a configured compiler on each machine	Runs in any browser; output in about a second
Distributing exercises	Printed manuals and messages	Faculty assign exercises; students see them with due dates
Submission	Files sent individually	One click; stored with status and history
Keeping a record of work	Local folders, easily lost	Automatically committed to the student's private GitHub repository
Evaluation	Paper or spreadsheet	Marks and feedback recorded against each submission
Attendance	Paper register	Timetable-aware, automatic, with percentage reports
Exams	Paper or separate software	Timed online exam with automatic grading and focus monitoring
Administration	Scattered records	Central console with audit log

4.5 Discussion

The results demonstrate that the objectives of the project were met. Students can write, run and submit programs from a browser; the in-browser IDE returns results quickly and reports errors clearly; submissions can be mirrored to GitHub without any manual Git operations; and faculty and administrators have the tools they need to run the laboratory term from the same platform.

Chapter 5
Conclusion and Future Scope

5.1 Conclusion

LabFlow brings the whole programming laboratory workflow of the MCA programme onto a single web platform. Its in-browser IDE, built on the Monaco editor and backed by a time-limited execution engine with a Judge0 fallback, lets students write, run and test C, Java and Python programs without any local set-up. Its GitHub synchronisation turns each submission into a commit in a private repository that the system creates and maintains for the student, so that laboratory work becomes a growing, version-controlled portfolio. Around these two features the system provides secure role-based access with Google sign-in, faculty tools for exercises, test cases, evaluation and feedback, timetable-based attendance, timed online exams with focus monitoring, and an administration console with announcements, audit log and maintenance mode.

The project also gave valuable practical experience in full-stack development with React, FastAPI and MongoDB, in OAuth-based integration with Google and GitHub, in secure handling of credentials and tokens, and in designing a service that safely runs code submitted by users. LabFlow is ready to support the laboratory term of the department and provides a solid base for further growth.

5.2 Future Scope

- Containerised execution: run every program inside a dedicated container with CPU, memory and network restrictions for stronger isolation, or self-host Judge0 for the whole workload.
- More languages: add C++, JavaScript, SQL and PL/SQL for the Advanced DBMS laboratory, and shell scripting for the Linux exercises of Network Security and Applications.
- Richer IDE: multi-file projects, saved files in the cloud, debugging support and real-time collaboration between a student and a faculty member.
- Deeper GitHub features: pull-request based review, README generation for each laboratory repository and a public portfolio page.
- Learning analytics: dashboards that show class performance, common errors and submission trends to help faculty plan teaching.
- Plagiarism checking: similarity detection across submissions of the same exercise.

- Notifications: e-mail and mobile push reminders for due dates, evaluations and upcoming laboratory sessions.
- Scaling to more departments: multi-department and multi-semester support with cloud deployment and automated backups.

Chapter 6
Coding

6.1 Source Code

This chapter lists the source code of the modules that implement the main features of LabFlow: the application entry point, authentication, the code execution engine, GitHub synchronisation, attendance, the exam focus guard and the in-browser editor. The complete source code is available in the project repository at https://github.com/AllenJohnn/labflow.

6.1.1 main.py

FastAPI application entry point (backend/app/main.py)

```
from contextlib import asynccontextmanager


from fastapi import FastAPI, Request
from fastapi.middleware.cors import CORSMiddleware
from starlette.middleware.sessions import SessionMiddleware

from app.config.settings import settings
from app.database.mongodb import check_database_connection
from app.database.indexes import create_indexes
from app.services.admin_service import init_default_admin
from app.services.faculty_service import init_default_faculty, init_default_lab_data
from app.services.student_service import init_default_student
from app.routes import health, auth, student, faculty, admin, exams

import os
import sys

from urllib.parse import urlparse

_debug_mode = os.getenv("DEBUG", "False").lower() in ("true", "1", "yes")
_mongodb_uri = os.getenv("MONGODB_URI", "")
_host = urlparse(_mongodb_uri).hostname or ""

if _debug_mode and _host not in ("localhost", "127.0.0.1"):
    print("CRITICAL ERROR: Refusing to boot with DEBUG enabled against a non-local MONGODB_URI.", file=sys.stderr)
    sys.exit(1)

async def async_db_init():
    try:
        await check_database_connection()
```

```
        await create_indexes()
        await init_default_admin()
        await init_default_faculty()
        await init_default_student()
        await init_default_lab_data()
        print("[Database] Initialization completed successfully.")
    except Exception as e:
        print(f"[Database] Startup initialization notice: {e}")

from app.database.mongodb import db_wrapper

@asynccontextmanager
async def lifespan(app: FastAPI):
    db_wrapper.connect()
    await async_db_init()
    yield
    await db_wrapper.close()

from fastapi.responses import JSONResponse
from app.services.admin_service import is_maintenance_active, get_system_settings

app = FastAPI(
    title="LabFlow API",
    description="Backend REST API for the LabFlow Programming Laboratory Management System.",
    version="1.0.0",
    lifespan=lifespan
)

@app.middleware("http")
async def maintenance_middleware(request: Request, call_next):
    path = request.url.path
    if (
        path.startswith("/api/v1/admin")
        or path.startswith("/api/v1/health")
        or path.startswith("/api/v1/auth")
        or path.startswith("/docs")
        or path.startswith("/openapi.json")
    ):
        return await call_next(request)

    if is_maintenance_active():
        sys_settings = await get_system_settings()
        if sys_settings.get("maintenance_mode", False):
            return JSONResponse(
```

```
            status_code=503,
            content={
                "status": "maintenance",
                "detail": sys_settings.get(
                    "maintenance_message",
                    "Maintenance in progress. The system is temporarily unavailable while maintenance is being performed. Please try again later."
                ),
                "expected_return": sys_settings.get("expected_return", "Shortly"),
                "maintenance_mode": True
            }
        )

    return await call_next(request)

app.add_middleware(
    SessionMiddleware,
    secret_key=settings.SESSION_SECRET_KEY
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        settings.FRONTEND_URL,
        "http://localhost:5173",
        "http://127.0.0.1:5173",
        "http://localhost:5174",
        "http://127.0.0.1:5174",
        "http://localhost:3000",
    ],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(
    health.router,
    prefix="/api/v1/health",
    tags=["Health"]
)

app.include_router(
    auth.router,
    prefix="/api/v1/auth",
    tags=["Authentication"]
```

```
)

app.include_router(
    student.router,
    prefix="/api/v1/student",
    tags=["Student"]
)

app.include_router(
    faculty.router,
    prefix="/api/v1/faculty",
    tags=["Faculty"]
)

app.include_router(
    admin.router,
    prefix="/api/v1/admin",
    tags=["Admin"]
)

app.include_router(exams.student_exam_router, prefix="/api/v1")
app.include_router(exams.faculty_exam_router, prefix="/api/v1")
```

6.1.2 jwt_service.py

JSON Web Token creation and verification (backend/app/services/jwt_service.py)

```
from datetime import datetime, timedelta, timezone
from jose import jwt

from app.config.settings import settings

def create_access_token(
    user_id: str,
    role: str = "student",
    google_id: str | None = None,
    name: str = "",
    email: str = "",
    picture: str = ""
):
    expire = datetime.now(timezone.utc) + timedelta(
        minutes=settings.JWT_EXPIRE_MINUTES
    )
```

```
  payload = {
    "sub": user_id,
    "role": role,
    "google_id": google_id,
    "name": name,
    "email": email,
    "picture": picture,
    "exp": expire
  }

  return jwt.encode(
    payload,
    settings.JWT_SECRET_KEY,
    algorithm=settings.JWT_ALGORITHM
  )

def decode_access_token(token: str) -> dict | None:
  try:
    payload = jwt.decode(
      token,
      settings.JWT_SECRET_KEY,
      algorithms=[settings.JWT_ALGORITHM]
    )
    return payload
  except Exception:
    return None
```

6.1.3 security.py

Password hashing with PBKDF2 (backend/app/core/security.py)

```
import hashlib
import os
import hmac

def hash_password(password: str) -> str:
  salt = os.urandom(16)
  key = hashlib.pbkdf2_hmac(
    'sha256',
    password.encode('utf-8'),
    salt,
    100000
  )
  return f"{salt.hex()}${key.hex()}"
```

```
def verify_password(password: str, hashed_password: str) -> bool:
  try:
    salt_hex, key_hex = hashed_password.split('$')
    salt = bytes.fromhex(salt_hex)
    expected_key = bytes.fromhex(key_hex)

    computed_key = hashlib.pbkdf2_hmac(
      'sha256',
      password.encode('utf-8'),
      salt,
      100000
    )
    return hmac.compare_digest(computed_key, expected_key)
  except Exception:
    return False
```

6.1.4 local_execution_service.py

Local code execution engine (backend/app/services/local_execution_service.py)

```
"""
LabFlow Local Execution Service

Runs code locally via subprocess for Python and Java.
Falls back to Judge0 for C (no local GCC) or when local tools are unavailable.
Provides near-instant execution vs. 15-30s Judge0 CE queue times.
"""

import asyncio
import logging
import os
import shutil
import subprocess
import tempfile
import time
from typing import Dict, Any

from app.config.settings import settings

logger = logging.getLogger(__name__)

# Detect locally available compilers/interpreters at startup
_python_bin = shutil.which("python") or shutil.which("python3")
```

```
_javac_bin = shutil.which("javac")
_java_bin = shutil.which("java")
_gcc_bin = shutil.which("gcc")

LOCAL_CAPABILITIES = {}
if _python_bin:
    LOCAL_CAPABILITIES["python"] = _python_bin
if _javac_bin and _java_bin:
    LOCAL_CAPABILITIES["java"] = (_javac_bin, _java_bin)
if _gcc_bin:
    LOCAL_CAPABILITIES["c"] = _gcc_bin

logger.info(f"[LocalExec] Detected capabilities: {list(LOCAL_CAPABILITIES.keys())}")



def _truncate(text: str, max_bytes: int = 0) -> str:
    """Truncate output to configured max bytes."""
    limit = max_bytes or getattr(settings, "IDE_MAX_OUTPUT_BYTES", 65536)
    if not text:
        return ""
    if len(text) > limit:
        return text[:limit] + "\n...[Output Truncated]"
    return text



def _make_result(
    status: str,
    execution_status: str,
    language: str,
    stdout: str = "",
    stderr: str = "",
    compile_output: str = "",
    exit_code: int = 0,
    execution_time_ms: int = 0,
    memory_kb: int = 0,
    timed_out: bool = False,
) -> Dict[str, Any]:
    """Build a normalized execution result dict."""
    return {
        "status": status,
        "execution_status": execution_status,
        "language": language,
        "stdout": _truncate(stdout),
        "stderr": _truncate(stderr),
```

Chapter 7

Screen Shots

The login page of LabFlow. Students and faculty continue with their institutional Google account; the Faculty Login and Admin Login links lead to the portals for those roles.

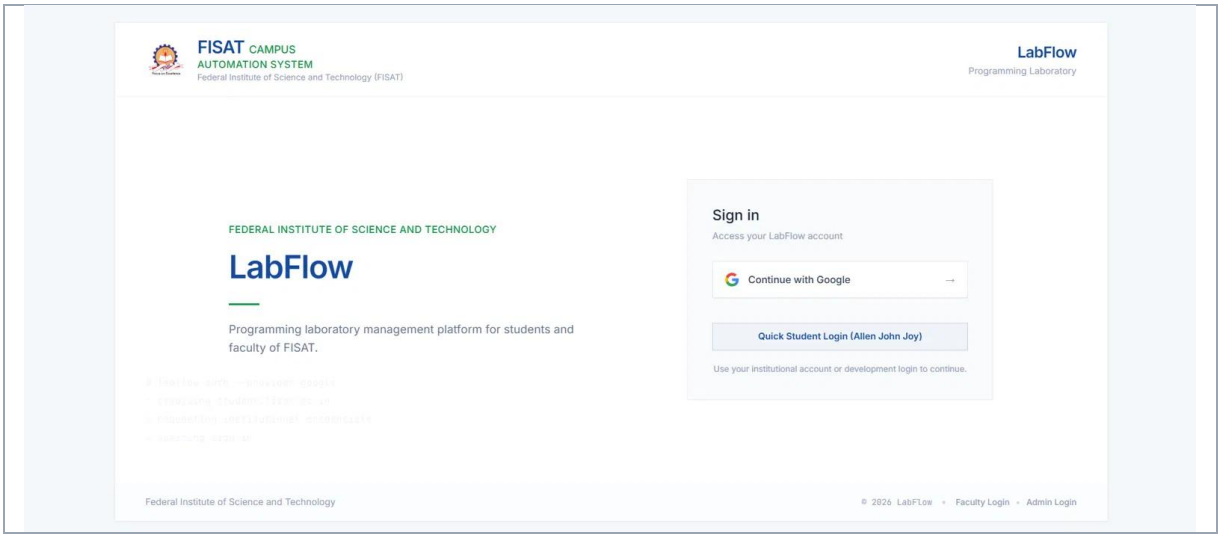


Figure 7.1: Login page with Google sign-in, quick student login for development, and Faculty and Admin login links

The student dashboard greets the student, shows the programme and semester, overall attendance and cards for each assigned laboratory with its faculty.

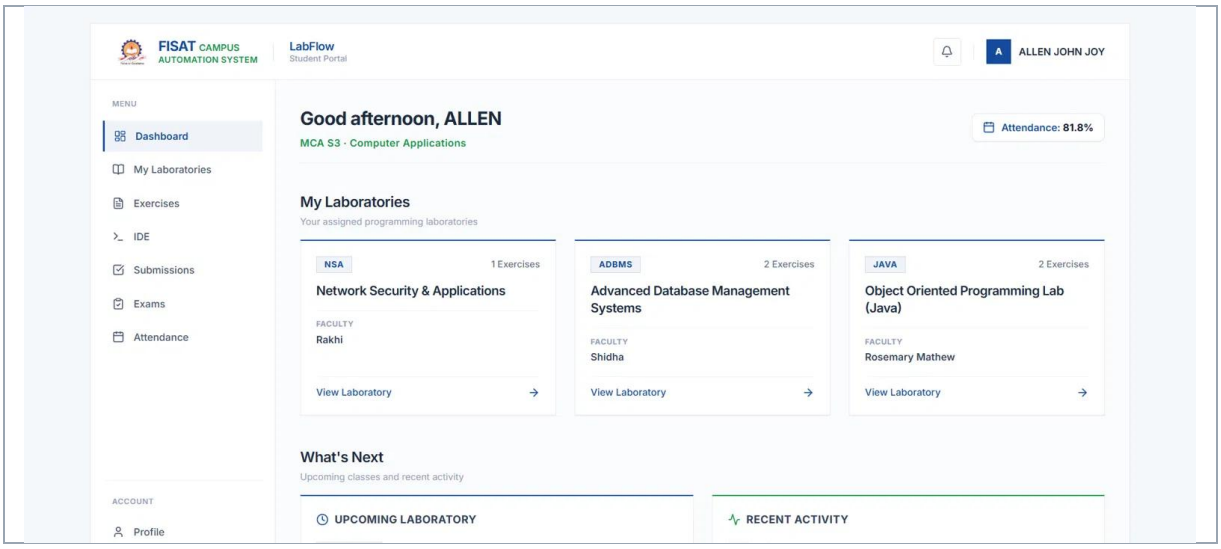


Figure 7.2: Student dashboard

A laboratory page lists the faculty, a link to the syllabus and the laboratory exercises with due dates and submission status.

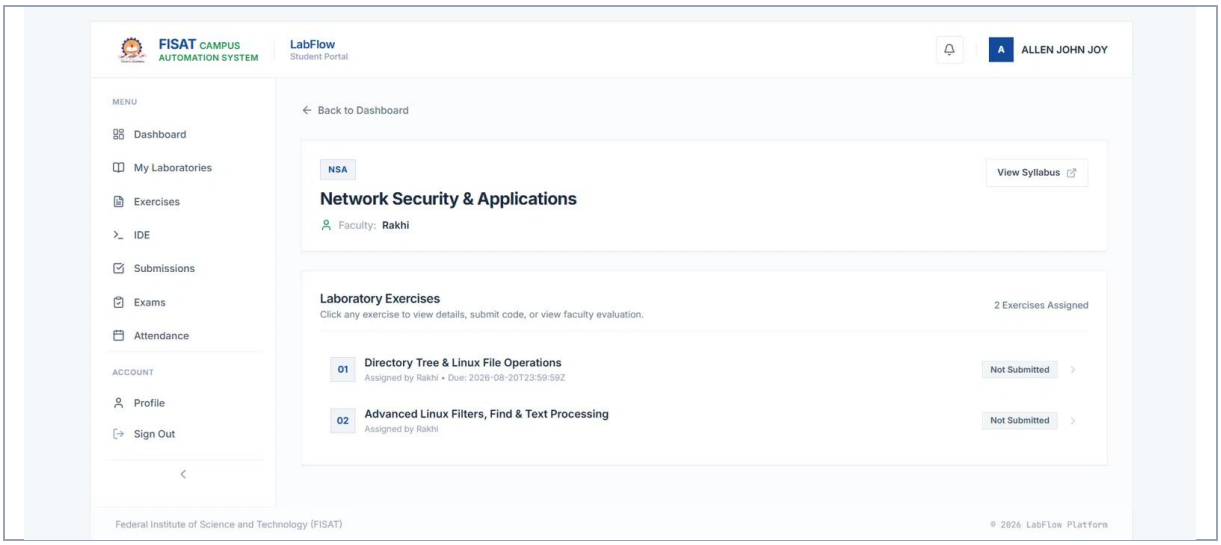


Figure 7.3: Laboratory page of Network Security and Applications (student view)

The Exercises page lists exercises from all laboratories and can be filtered by course.

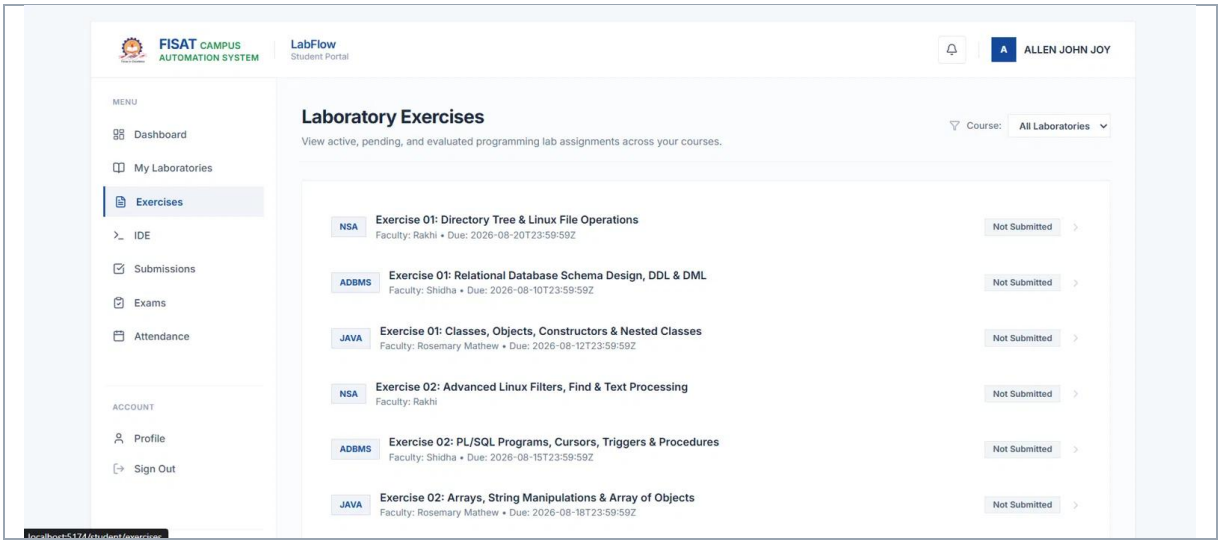


Figure 7.4: Laboratory Exercises page with course filter

The in-browser IDE: the Monaco editor on the left, language selector and Run button at the top, and the Output, Input and Errors tabs with a SUCCESS status badge on the right. The program prints 150. A Standard Input panel is available at the bottom.

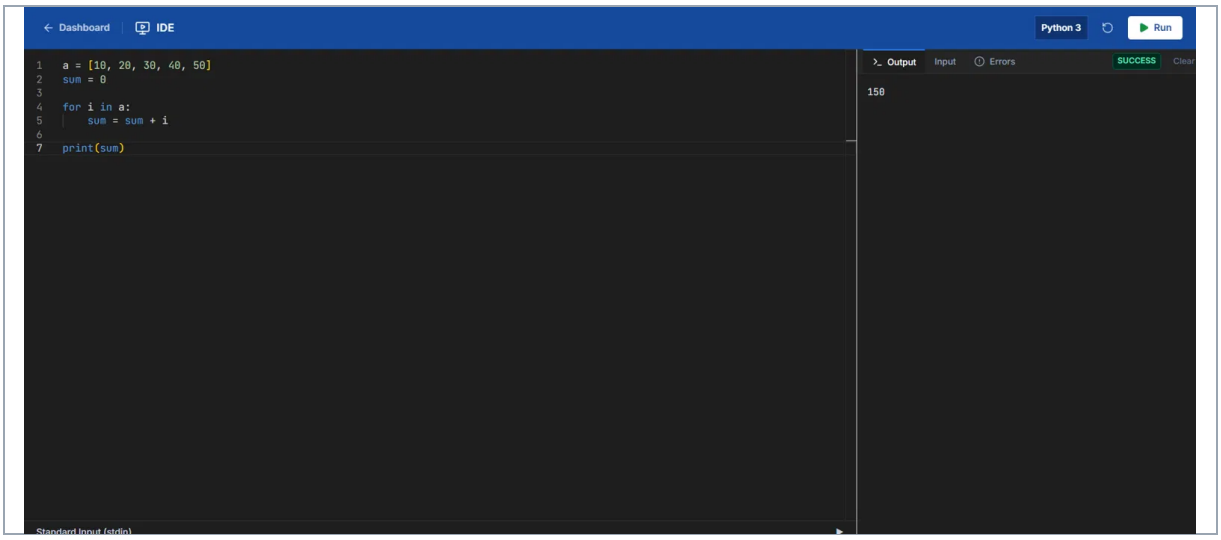


Figure 7.5: In-browser IDE running a Python program

The Attendance page shows overall attendance, sessions attended, the next laboratory session and a per-laboratory breakdown.

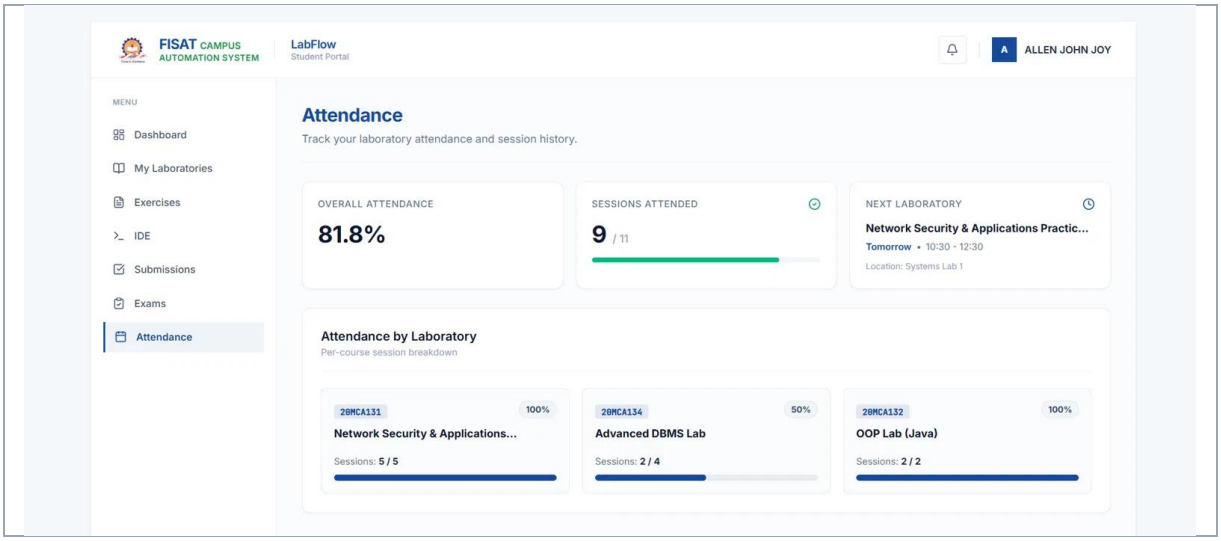


Figure 7.6: Student attendance page

The faculty dashboard summarises laboratories, enrolled students, submissions received and submissions pending review, with a Needs Attention list.

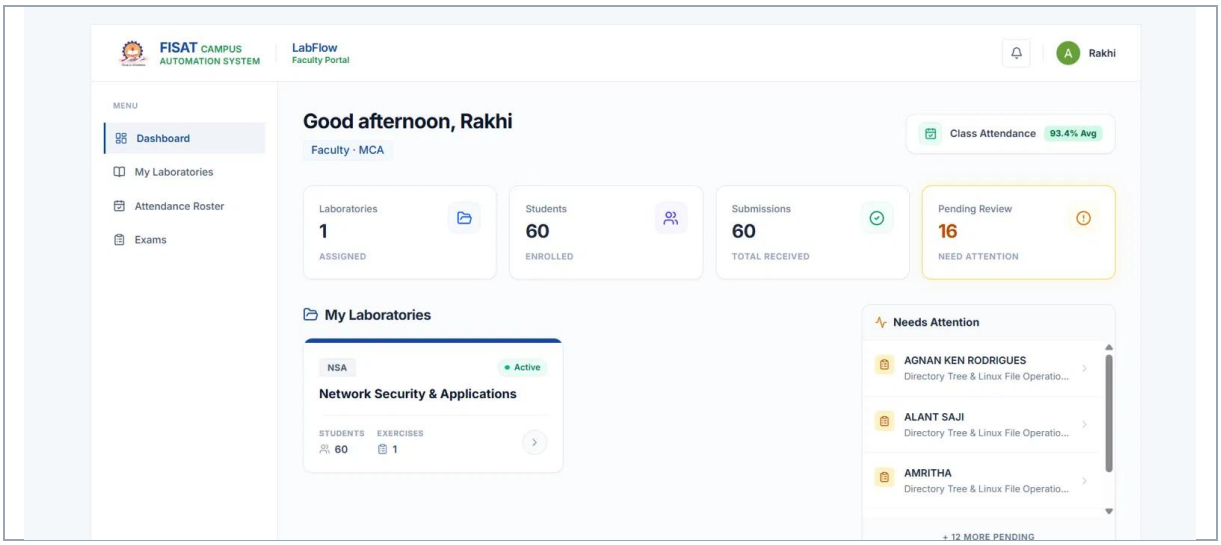


Figure 7.7: Faculty dashboard

The faculty laboratory page has tabs for Overview, Exercises, Submissions, Students and Announcements, and allows the syllabus to be viewed and updated.

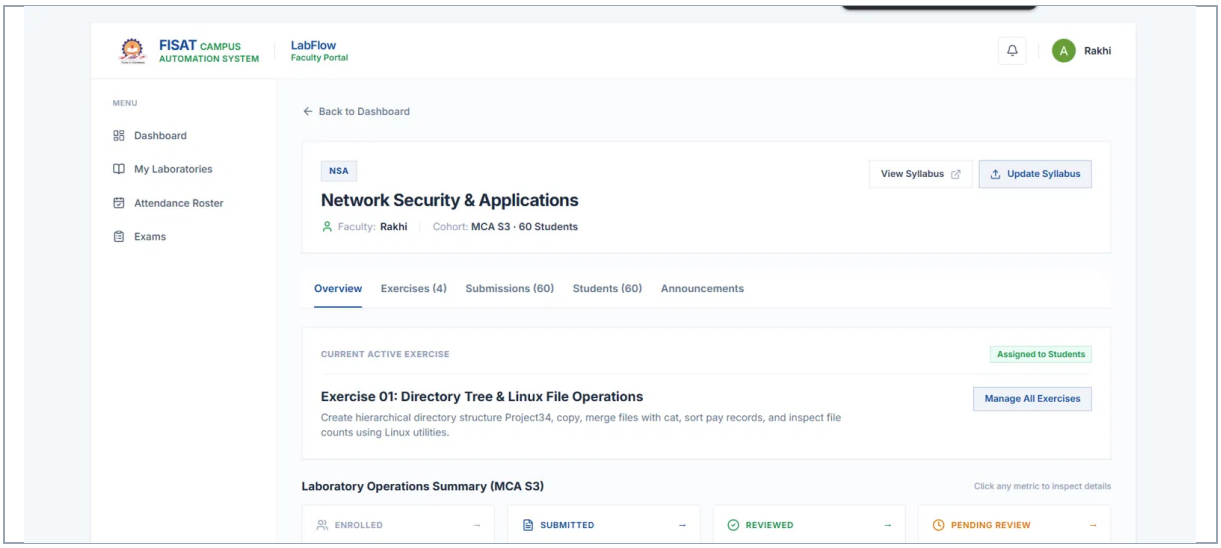


Figure 7.8: Faculty laboratory page

The exam studio lets faculty create and publish a timed assessment, add coding and multiple-choice questions and review student attempts.

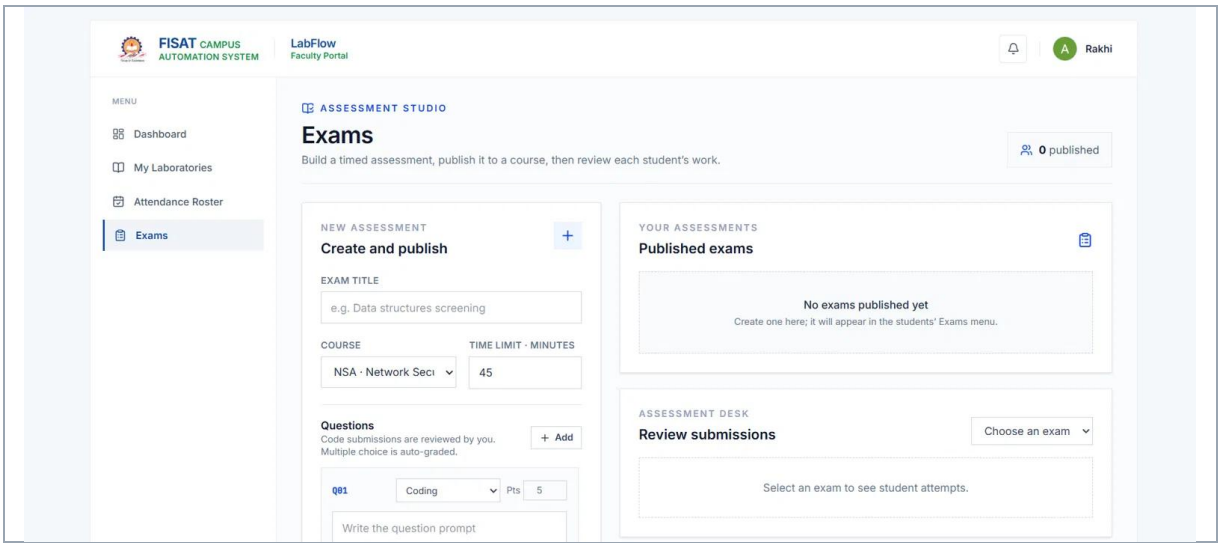


Figure 7.9: Faculty Exams (Assessment Studio)

Administrators publish announcements to students, faculty or everyone and can delete old notices.

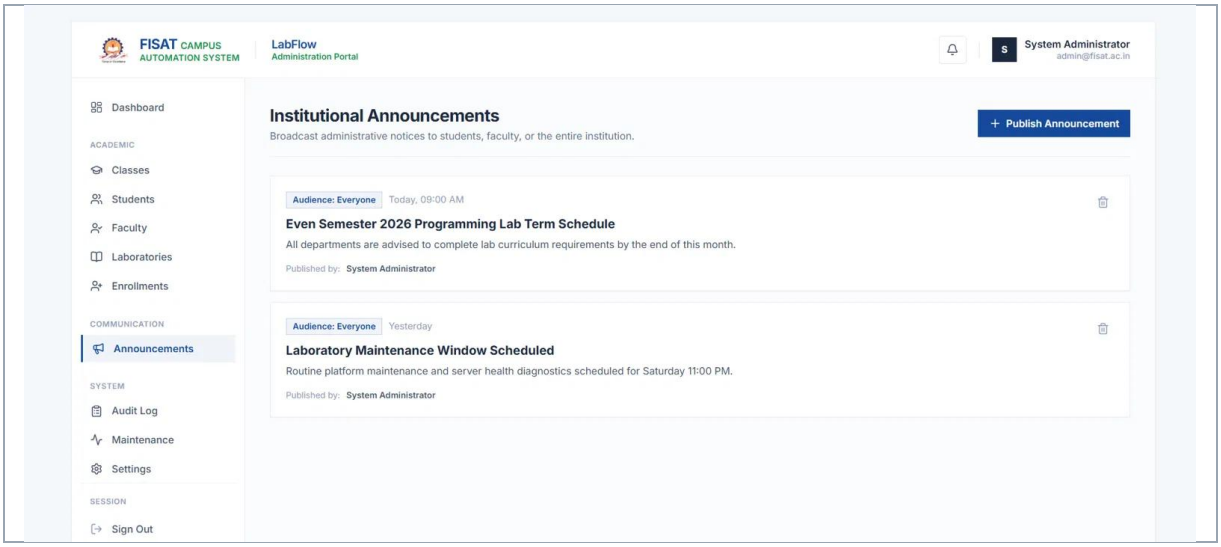


Figure 7.10: Administrator: institutional announcements

The administration dashboard shows system status, key counts and the academic programmes and class sections.

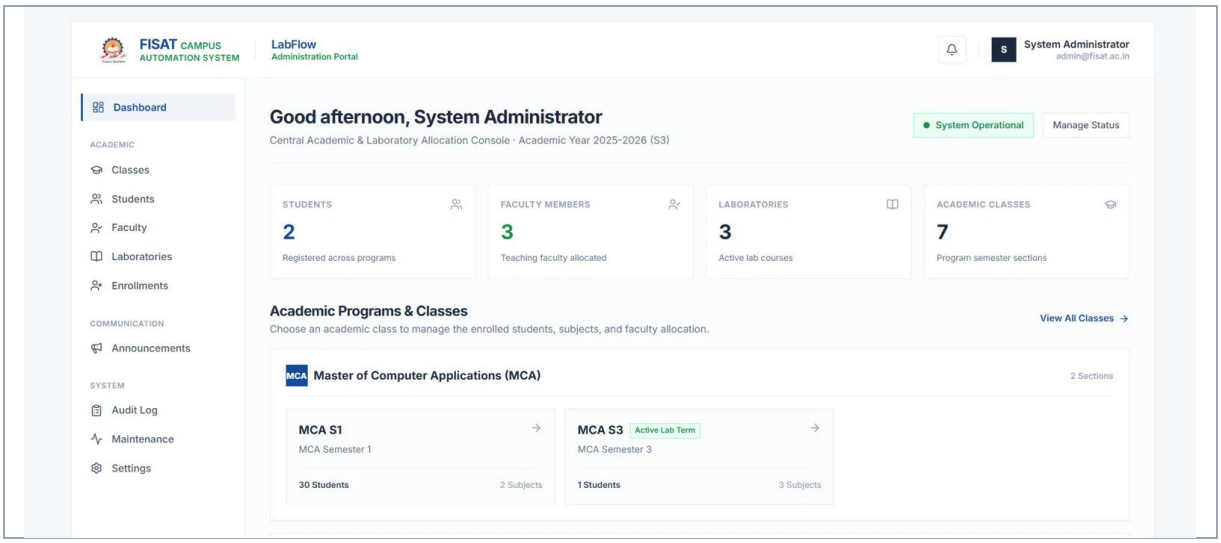


Figure 7.11: Administrator dashboard

Chapter 8
References

[1] P. Ihantola, T. Ahoniemi, V. Karavirta and O. Seppälä, “Review of recent systems for automatic assessment of programming assignments,” in Proc. 10th Koli Calling Int. Conf. on Computing Education Research, 2010, pp. 86–93.

[2] C. Douce, D. Livingstone and J. Orwell, “Automatic test-based assessment of programming: A review,” ACM Journal on Educational Resources in Computing, vol. 5, no. 3, Art. 4, 2005.

[3] D. Hardt, Ed., “The OAuth 2.0 Authorization Framework,” RFC 6749, Internet Engineering Task Force, Oct. 2012.

[4] M. Jones, J. Bradley and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, Internet Engineering Task Force, May 2015.

[5] R. T. Fielding, “Architectural Styles and the Design of Network-based Software Architectures,” Ph.D. dissertation, Univ. of California, Irvine, 2000.

[6] K. Moriarty, B. Kaliski and A. Rusch, “PKCS #5: Password-Based Cryptography Specification Version 2.1,” RFC 8018, Internet Engineering Task Force, Jan. 2017.

[7] K. Schwaber and J. Sutherland, “The Scrum Guide,” Nov. 2020. [Online]. Available: https://scrumguides.org

[8] FastAPI documentation. [Online]. Available: https://fastapi.tiangolo.com

[9] MongoDB Manual. [Online]. Available: https://www.mongodb.com/docs/manual/

[10] React documentation. [Online]. Available: https://react.dev

[11] Monaco Editor, Microsoft. [Online]. Available: https://microsoft.github.io/monaco-editor/

[12] Judge0 – open-source online code execution system. [Online]. Available: https://judge0.com

[13] GitHub REST API documentation. [Online]. Available: https://docs.github.com/en/rest

[14] Fernet (symmetric encryption), Python cryptography documentation. [Online]. Available: https://cryptography.io/en/latest/fernet/

[15] Authlib documentation. [Online]. Available: https://docs.authlib.org

[16] Vite documentation. [Online]. Available: https://vite.dev

[17] Google Identity, “OpenID Connect.” [Online]. Available: https://developers.google.com/identity/openid-connect/openid-connect